

Project Report

AI-Driven Attack Defense for IoT Smart City in 6G Networks

Prepared by:

Kaouech Maram

Ben Hamida Mohamed Khaled

Fatnassi Hadil

Ouarag Mohamed Seifeddine

Bensethom Nawres

Trabelsi Amine

Class: 4 DATA

Contents

1	Scientific Context	5
1.1	How a Smart City Works (system view)	5
2	Problem Formulation: Study & Critique of Existing Approaches	9
2.1	The Gap in Existing Solutions	9
2.2	Central Problem Statement	10
2.3	Where the Threats Appear in a 6G/5G Smart City (attack surface)	10
2.4	Study and Critique of Existing Approaches	11
2.5	Resulting Gap and Project Direction	12
3	Business Objectives	16
4	Data Science Objectives	18
5	Identification of Decision-Makers	20
6	CRISP-DM Methodology	21
7	SDG Objectives	22
7.1	Relevant SDGs and Alignment with the Project	22
8	Data Understanding and Exploratory Data Analysis	25
8.1	6G IoT Smart City Dataset	25
8.1.1	Dataset Structure	25
8.1.2	Feature Categories (Official Grouping)	26
8.1.3	Missing Values	27
8.1.4	Label Distribution	27
8.1.5	Correlation Analysis	28
8.1.6	Behavioral Mapping of Attacks (6G)	29
8.2	5G Smart City Network Slicing Datasets	30
8.2.1	Dataset Dimensions	30
8.2.2	Feature Categories (Official Grouping)	30

8.2.3	Missing Values	31
8.2.4	Label Distribution	32
8.3	Outcomes of Preprocessing	32
8.3.1	5G Dataset (Global) – Feature Engineering	32
8.3.2	6G Dataset – Feature Breakdown	32
8.3.3	Impact & Readiness	33
8.4	Modelling Phase – 6G Network Intrusion Detection	33
8.4.1	Experimental Design	33
8.4.2	Feature Engineering	34
8.4.3	Model Selection	34
8.4.4	Isolation Forest Model	34
8.4.5	Autoencoder Model	35
8.4.6	Evaluation Metrics	35
8.4.7	Results	36
8.4.8	Threshold Optimisation	36
8.4.9	Discussion	36
8.5	Modelling Phase – 5G Network Intrusion Detection	37
8.5.1	Experimental Design	37
8.5.2	Machine Learning Models	37
8.5.3	Confusion Matrix Interpretation	37
8.5.4	ROC and Precision-Recall Curves	38
8.5.5	Model Comparison and Results	38
8.5.6	Conclusion of the Modelling Phase	38
8.6	Model Architecture – Mixture-of-Experts Ensemble	39
9	MLOps Pipeline	41
9.1	Introduction to MLOps	41
9.1.1	Why MLOps?	41
9.1.2	Pillars of MLOps	42
9.1.3	CI/CD for Machine Learning	42
9.2	Global Pipeline Architecture	42
9.3	CI/CD Pipeline – Results & Coverage	43
9.4	Data Versioning with DVC	43
9.4.1	Problem Statement	43
9.4.2	DVC Architecture	44
9.5	Experiment Tracking with MLflow	44
9.6	Model Registry	46
9.7	Automated Training Pipeline	46
9.8	CI/CD with GitHub Actions	47

9.9 Dockerization	48
10 Application Architecture & Technology Stack	51
10.1 Application Architecture	51
10.2 Technology Stack	52
11 Deployment & API	53
11.1 Deployment Architecture	53
11.2 FastAPI Service	53
11.2.1 API Endpoints	55
11.3 Model Serving – Real-Time Inference	55
11.4 Docker Deployment	56
11.5 Scalability Considerations	57
12 Dashboard & Monitoring	58
12.1 Dashboard Features	58
12.2 Monitoring Objectives	58
12.3 Grafana Dashboard	59
12.4 Prometheus Integration	60
12.5 Alerting System	61
12.6 Real-Time Threat Visualization	61
13 Trustworthy AI Principles	62
14 Testing & Validation	63
14.1 Functional Testing	63
14.2 Performance Testing	64
14.3 Scalability Testing	64
14.4 Security Testing	65
14.5 Stress Testing	65
15 Results & Discussion	66
15.1 Final Results	66
15.1.1 Model Performance Comparison	66
15.1.2 MLOps Pipeline KPIs	67
15.2 Value Proposition – VERADO IDS	67
15.3 Business Impact	68
15.4 Technical Impact	68
15.5 Limitations	69
15.6 Lessons Learned	69

16 Conclusion & Future Work	70
16.1 Conclusion	70
16.2 Future Improvements	70
16.2.1 Federated Learning	70
16.2.2 Explainable AI (XAI)	71
16.2.3 Zero Trust Network Security	71
16.2.4 Edge AI	71
16.2.5 Full Autonomous Cyber Defense	71
A Code Snippets	73
A.1 Preprocessing Script	73
A.2 Training Script with MLflow	74
B Configuration Files	76
B.1 Dockerfile	76
B.2 docker-compose.yml	76
C API Documentation	78
D Dataset Description	79

Chapter 1

Scientific Context

Smart Cities represent a major evolution in modern urban development, aiming to improve citizens' quality of life through the intelligent use of digital technologies. By integrating ICT, IoT, cloud/edge computing, big data analytics, and AI, Smart Cities enable more efficient management of urban resources and services (transportation, energy, safety, healthcare, environment, etc.).

IoT devices such as sensors, cameras, smart meters, and connected vehicles continuously generate massive volumes of heterogeneous data. These data are transmitted over advanced communication infrastructures, including emerging sixth-generation (6G) networks, which target ultra-low latency, high data rates, massive connectivity, and stronger AI-native automation.

However, deploying IoT at city scale significantly increases the attack surface. Cyberattacks targeting IoT and network infrastructures can disrupt critical services, compromise data, and threaten public safety. Ensuring cybersecurity in Smart City environments is therefore a major scientific and technological challenge.

The convergence of IoT, 6G networks, and AI-driven analytics accelerates Smart City innovation while introducing complex cybersecurity challenges that require intelligent, automated, scalable, and near real-time detection and defense mechanisms.

1.1 How a Smart City Works (system view)

Figure 1.1 illustrates a four-layer smart city architecture adapted from Zhao and Zhang (2020). The architecture is structured as follows: (i) the **Perceptual layer**, which includes sensors, cameras, user devices, and intelligent terminals that collect real-world data; (ii)

the **Network layer**, responsible for transmitting data through communication networks and the Internet; (iii) the **Platform layer**, which manages data storage, integration, standards, and modeling; and (iv) the **Application layer**, where smart services such as smart government, transportation, healthcare, smart economy, manufacturing, safe city, smart community, and citizen card are delivered.

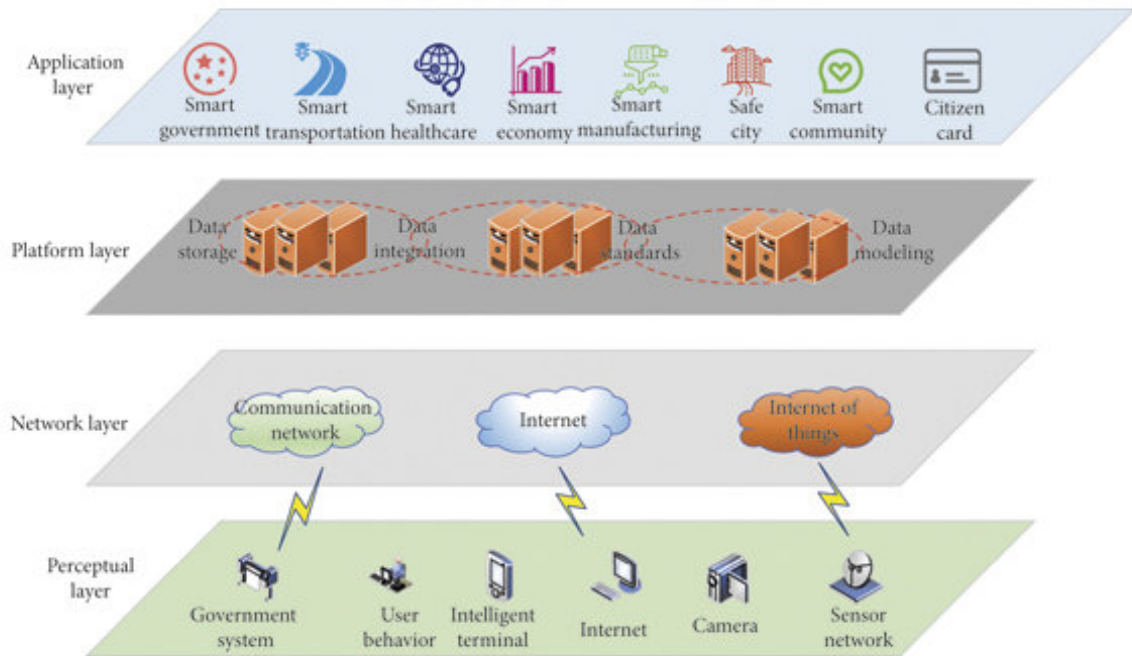


Figure 1.1: Four-layer Smart City architecture (Perceptual, Network, Platform, and Application layers). Adapted from: Zhao, Z., & Zhang, Y. (2020). *Impact of Smart City Planning and Construction on Economic and Social Benefits Based on Big Data Analysis*. Complexity, 2020, 1–11.

1.2 Comparison of 5G and 6G Networks for Smart Cities

Figure 1.2 presents a comparative overview of 5G and 6G network architectures, highlighting the structural evolution from traditional cloud-centric communication systems to intelligent, AI-native, and edge-enabled infrastructures.

1. 5G Network Architecture

On the left side of the figure, the 5G architecture is organized into three main planes:

User Plane: Represents end-user devices such as smartphones, vehicles, and IoT devices generating data traffic.

Access Plane: Includes radio access technologies such as LTE, GSM, Wi-Fi, NR, and other wireless standards. This layer connects user devices to the core network.

Core Plane: The core network relies mainly on:

- Cloud and IP computing
- Centralized data processing
- Gateway functions
- Billing systems

In 5G, intelligence is primarily centralized in the cloud, and although network slicing exists, decision-making remains largely centralized.

2. 6G Network Architecture

On the right side of the figure, the 6G architecture introduces a transformation toward an AI-native and intelligent infrastructure.

Intelligent User Plane: Supports advanced devices such as industrial IoT, healthcare IoT, autonomous vehicles, and smart environments, generating higher data volumes.

Intelligent Access Plane: Integrates:

- Multi-access technologies (UWB, Wi-Fi, LTE, NR, 6G)
- Edge computing (MEC)
- Fog computing
- AI-driven traffic management

Unlike 5G, intelligence is pushed closer to the edge to reduce latency and enable real-time decisions.

Intelligent Core Plane: Incorporates:

- Intelligent gateway core
- AI-driven orchestration
- Automated billing systems
- Cloud and IP computing integrated with AI

3. Key Architectural Differences

The main differences between 5G and 6G include:

- 5G is primarily cloud-centric, while 6G is edge-intelligent and AI-native.
- 6G tightly integrates computing and communication (MEC, Fog, AI).
- 6G enables ultra-low latency and extreme automation.
- 6G supports massive IoT and mission-critical applications.

This architectural evolution increases network complexity, attack surface, and dependency on AI-based automation. Therefore, cybersecurity mechanisms in 6G Smart Cities must be scalable, real-time, AI-driven, and context-aware. This comparison justifies the need for intelligent, flow-based detection systems adapted to next-generation 6G infrastructures.

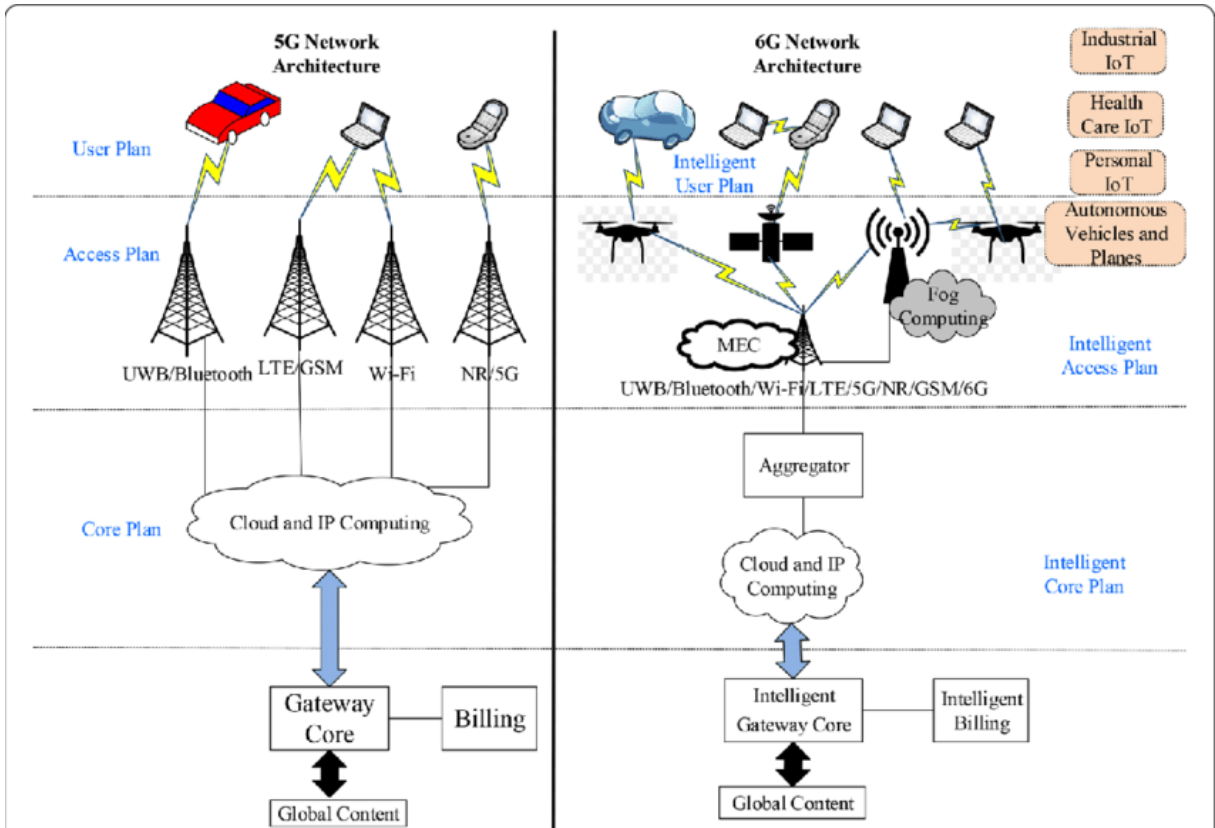


Figure 1.2: Comparative architecture of 5G and 6G networks. 6G introduces intelligent user, access, and core planes with integrated AI, edge (MEC), and fog computing (Adapted from [2]).

Chapter 2

Problem Formulation: Study & Critique of Existing Approaches

2.1 The Gap in Existing Solutions

Existing intrusion detection and network security solutions present several critical limitations in the context of 5G/6G Smart City environments:

- **Lack of AI-based behavioral detection for unknown attacks:** Traditional systems rely on signatures and rules, failing to detect zero-day or variant threats.
- **Not optimized for 5G/6G network slicing and massive IoT traffic:** Existing tools are not designed for the heterogeneous, slice-based traffic patterns of next-generation networks.
- **Limited real-time intelligence and predictive capability:** Most solutions are reactive rather than proactive, lacking automated and adaptive response mechanisms.
- **High cost, low transparency, and poor explainability in enterprise tools:** Commercial solutions often operate as black boxes, making them difficult to audit or justify to operators.

Our Solution

Our project fills this gap with an AI-powered, behavior-based intrusion detection system tailored for Smart City 5G/6G environments, providing:

- AI-driven, explainable detection for unknown threats
- 5G/6G-optimized slice modeling with IoT scalability
- Real-time, scalable MLOps pipeline with automated retraining

- Transparent, cost-efficient, and interpretable system for operators

2.2 Central Problem Statement

Smart City networks are highly distributed and heterogeneous, combining large numbers of IoT devices (often resource-constrained), mobile users, and latency-critical services. In 5G/6G-style deployments, traffic is also separated into service contexts/slices (e.g., eMBB, mMTC, URLLC), where “normal” behavior differs significantly across services.

In this setting, attacks such as DDoS/botnets, scanning, spoofing, malware-driven abnormal traffic, and resource exhaustion can emerge at multiple entry points and propagate quickly. Classic defenses that rely on static rules, signatures, or centralized analysis struggle to:

- **Keep up with scale:** massive numbers of flows and continuous traffic make heavy-weight inspection difficult.
- **Handle evolving threats:** variants and previously unseen device behaviors reduce the effectiveness of signature-only methods.
- **Operate close to real time:** low-latency services (URLLC) require fast detection decisions with minimal overhead.
- **Work under heterogeneous traffic:** different access locations and device clusters create non-uniform (non-IID) traffic distributions, making a single global detector unreliable at all points.

How can we design a flow-based, AI-driven detection system for Smart City traffic over 5G/6G-style infrastructures that (i) detects attacks including variants, (ii) remains scalable for high-throughput user-plane monitoring, (iii) is robust to heterogeneous traffic across locations and services, and (iv) supports near real-time operation with low false-alarm rates?

2.3 Where the Threats Appear in a 6G/5G Smart City (attack surface)

- **IoT/UE layer:** compromised devices generating botnet/DDoS traffic, abnormal scanning behavior, credential abuse.
- **Access / entrance points:** large volumes of IoT traffic enter through multiple access

locations; distributed attacks may be visible only when considering multiple points together.

- **Edge/MEC:** attacks against edge services (IoT brokers, analytics), resource exhaustion, poisoning of local monitoring/analytics pipelines.
- **Core user-plane (UPF/N6) and service functions:** abuse of user-plane capacity, abnormal flow patterns, and service/API misuse.
- **Slicing/virtualization:** slice resource exhaustion, misconfiguration risks, and isolation failures that can amplify the impact of abnormal traffic.

2.4 Study and Critique of Existing Approaches

This section reviews major categories of intrusion detection approaches for Smart City and 6G environments. The objective is not to implement all these approaches, but to justify the methodological choice adopted in this project.

A) Signature / Rule-Based IDS/IPS

Description: Traditional IDS/IPS systems detect attacks using predefined signatures, static rules, and policy thresholds.

Relevance to our project: These methods are widely deployed in operational networks but are not sufficient for detecting new or evolving attacks.

Why we do not adopt this approach: They cannot generalize to unseen attack variants and are not adaptive to dynamic Smart City traffic patterns.

B) Payload Inspection (DPI-Based Security)

Description: Deep Packet Inspection analyzes packet payloads to identify malicious patterns or application-level attacks.

Relevance to our project: DPI is effective when payload visibility exists.

Why we do not adopt this approach: In modern 5G/6G networks, most traffic is encrypted, making payload inspection impractical. Additionally, DPI is computationally expensive and unsuitable for scalable Smart City deployment.

C) Centralized ML/DL-Based Intrusion Detection

Description: Machine learning and deep learning models are trained centrally using aggregated flow data to classify traffic as benign or malicious.

Relevance to our project: This approach aligns directly with the project objective, which requires training ML/DL models using the provided dataset.

Limitations: Centralized models may face scalability and generalization challenges in real-world distributed deployments. However, for this academic project, centralized supervised learning on flow-based features is the most appropriate and feasible approach.

D) Federated Learning and Personalized Distributed Detection

Description: Federated Learning allows distributed training across multiple nodes without sharing raw data. Personalized FL extends this idea by adapting models to heterogeneous traffic at each edge location.

Relevance to our project: These approaches are highly relevant for real-world 6G Smart City deployments.

Why we do not implement them in this project: The provided dataset is centralized and does not simulate multiple distributed edge nodes. Implementing full FL infrastructure is beyond the scope of this project.

Chosen Approach for This Project

Based on the analysis above and the project requirements, we adopt:

- Flow-based traffic modeling (no payload inspection),
- Centralized supervised ML/DL training,
- Performance evaluation on test data using multiple metrics.

The goal is to develop a scalable and behavior-based detection model that can identify attacks in IoT Smart City traffic over 6G-style networks, while remaining consistent with the academic project constraints.

2.5 Resulting Gap and Project Direction

From the critique above, the key gap is a deployable detection approach that is **behavior-based, scalable, and operationally usable** for Smart City user-plane monitoring. Therefore, this project focuses on:

- **Flow-based detection at user-plane monitoring points (UPF/MEC):** use statistical flow features (rates, timing, flags, directionality) that remain available even when payload is encrypted.
- **Context-aware detection across services (eMBB/mMTC/URLLC):** reduce false alarms by modeling “normal” behavior per service context and detecting deviations/attacks relative to that context.
- **Operational outputs:** produce risk scoring and prioritized alerts with simple feature-based explanations to support faster response.
- **Robustness over time:** monitor data drift and update the model as Smart City traffic evolves.

Common vulnerabilities and cyber-attack types in IoT-based Smart City networks

Figure 2.1 illustrates a multi-layer Smart City architecture and highlights the main cyber-attack surfaces across the infrastructure. The diagram presents the ecosystem from IoT devices at the bottom layer to Cloud and AI services at the top layer, showing how vulnerabilities can emerge at different points in the system.

1. IoT Device Layer

At the lowest level, IoT devices such as smart meters, connected vehicles, sensors, and cameras generate continuous data streams. This layer is highly exposed due to:

- Device spoofing
- Botnet recruitment
- Compromised firmware
- Unauthorized access

Since IoT devices are often resource-constrained and widely distributed, they represent a primary entry point for attackers.

2. RAN / Access Network (5G/6G)

The Radio Access Network connects IoT devices to the core infrastructure. Attacks at this level include:

- Spoofing of base stations

- Signal manipulation
- Unauthorized access attempts

Because this layer handles massive connectivity, vulnerabilities can propagate rapidly to higher layers.

3. Network Slicing and Isolation

In 5G/6G environments, network slicing separates services (eMBB, mMTC, URLLC). However, the figure highlights:

- Slice-based attacks
- Isolation failures
- Resource exhaustion

If slice isolation is compromised, an attack targeting one service may impact others, including critical services.

4. Core Network and User Plane (UPF/SBA)

The core network manages routing, authentication, and traffic control. The figure shows potential threats such as:

- Service abuse
- UPF attacks
- Signaling overload
- Resource exhaustion

Attacks at this level can disrupt large portions of Smart City services simultaneously.

5. Edge and Cloud / AI Services

At the highest layer, data is processed in Edge Computing (MEC) and Cloud platforms. Vulnerabilities include:

- Data poisoning
- AI model manipulation
- Service abuse

- Infrastructure exhaustion

Because AI systems increasingly automate decisions in Smart Cities, compromising this layer can have large-scale operational consequences.

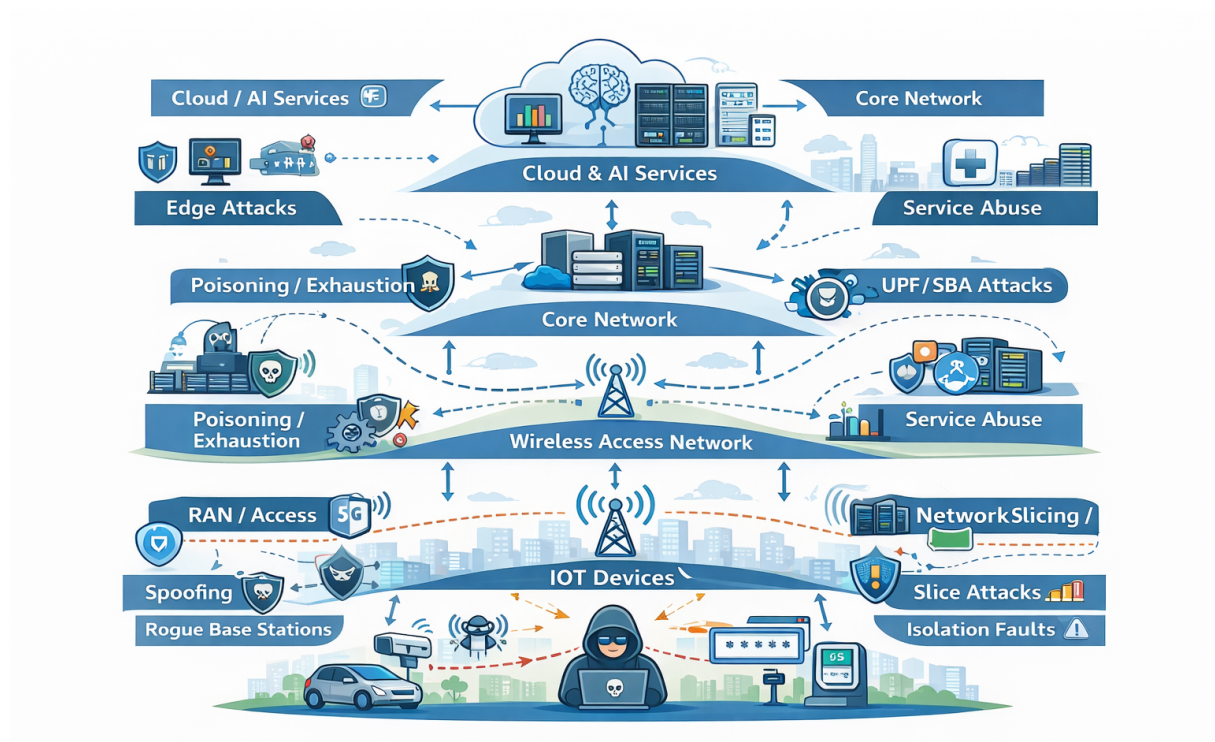


Figure 2.1: Common vulnerabilities and cyber-attack types in IoT-based Smart City networks.

Chapter 3

Business Objectives

This project aims to protect Smart City digital services (transportation, public safety, energy, healthcare, and environment monitoring) from cyber threats by reducing successful attacks, accelerating response, and lowering the day-to-day operational effort needed to stay protected as services scale.

BO1 – Service Continuity

Goal: Ensure that high-priority Smart City clients (such as healthcare institutions, emergency services, and public safety authorities) remain fully operational and protected, even during cyber-attacks or abnormal network conditions.

KPIs:

- Availability rate for priority clients during incident scenarios.
- Percentage of priority clients unaffected during network attacks.
- Performance degradation experienced by priority clients under abnormal traffic.
- Average time to fully restore service quality for priority clients after disruption.

BO2 – Operational Efficiency

Goal: Provide clear, prioritized early warnings that operators can trust, enabling faster actions while minimizing time wasted on false alarms.

KPIs:

- Operator false-alarm rate (target: decrease vs baseline).
- Average time between an early warning and operator action (target: decrease).

- Percentage of warnings that are confirmed as real issues after review (target: increase).

BO3 – Scalable Protection

Goal: Protect the network at large scale without heavy manual work, and keep protection effective as services expand and usage patterns change.

KPIs:

- Reduction in monthly human effort spent on manual updates and investigations (hours/month).
- Time required to roll out an improvement to detection logic (target: decrease).
- Coverage of monitored services and connections (target: increase while maintaining stable operations).

Chapter 4

Data Science Objectives

Each Business Objective is translated into one or more Data Science Objectives to ensure traceability between stakeholder value and technical work.

BO1 – Service Continuity

- **DS1.1 (Behavior-Based Threat Detection):** Train a behavior-based flow-level detection model for real-time traffic analysis and zero-day detection targeting high-priority Smart City services (healthcare, emergency, public safety).
- **DS1.2:** Enable early attack detection using short observation windows to trigger rapid mitigation for critical services.
- **DS1.3:** Model traffic behavior separately for each service slice (eMBB, mMTC, URLLC) and apply adaptive slice-specific detection.
- **DS1.4:** Generate risk scores and service-impact indicators to monitor availability and performance risks for priority clients.

BO2 – Operational Efficiency

- **DS2.1 (Precision and Explainable Alerts):** Reduce false alarms by tuning the model for precision while maintaining strong attack recall on imbalanced data, producing threat scoring outputs.
- **DS2.2:** Produce a calibrated risk score and alert severity levels (high/medium/low) to prioritize operator actions, with explainable alerts per prediction.
- **DS2.3:** Provide simple alert explanations by returning the top contributing flow features for each prediction.

BO3 – Scalable Protection

- **DS3.1 (Scalable, Maintainable MLOps Pipeline):** Deploy a lightweight, scalable inference pipeline based on flow statistics suitable for high-throughput environments (UPF/MEC), with automated CI/CD and containerized services.
- **DS3.2:** Monitor data drift and performance drift to detect when traffic behavior changes and retraining is needed.
- **DS3.3:** Version datasets and models and compare candidates against a baseline to safely update detection over time.

Chapter 5

Identification of Decision-Makers

The AI detection system receives inputs from network traffic (5G/6G, IoT, Slices) and logs & events (Firewall, IDS/IPS). The outputs feed several categories of stakeholders who use them to make operational decisions.

Primary Decision Makers

1. **Network Security Administrators** — Monitor anomalies and take mitigation actions in response to detected threats.
2. **Telecom Operators / ISPs** — Use detection outputs to reduce downtime and optimize network infrastructure.
3. **SOC Analysts (Security Operations Center)** — Analyze attack patterns and investigate suspicious traffic flows.

Secondary Stakeholders

4. **Company Management** — Track system performance and make strategic decisions based on security KPIs.
5. **Data/ML Engineers** — Monitor model drift and maintain the MLOps pipeline.

Chapter 6

CRISP-DM Methodology

This project follows the CRISP-DM (Cross-Industry Standard Process for Data Mining) methodology, an iterative and adaptive framework for data-driven projects.

- **Business Understanding:** Defined IDS detection goals, KPIs, and system requirements aligned with Smart City cybersecurity needs.
- **Data Understanding:** Explored 5G and 6G network traffic datasets and assessed data quality through comprehensive EDA.
- **Data Preparation:** Cleaned, normalized, and engineered features to produce model-ready datasets.
- **Modeling:** Trained and tuned Mixture-of-Experts (MoE) ensemble models combining XGBoost experts and Autoencoder anomaly detectors.
- **Evaluation:** Assessed model performance with key metrics (accuracy, F1, ROC-AUC, PR-AUC).
- **Deployment:** Containerized the full platform with a complete MLOps pipeline.

Chapter 7

SDG Objectives

This project contributes to the United Nations Sustainable Development Goals (SDGs) by strengthening the security and reliability of digital infrastructures that support Smart City services. Cybersecurity is a key enabler of safe, resilient, and trusted urban systems.

7.1 Relevant SDGs and Alignment with the Project

SDG 8 – Decent Work and Economic Growth

What it means: SDG 8 promotes sustained, inclusive, and sustainable economic growth, full and productive employment, and decent work. Reliable digital infrastructures are essential to support modern economic activities and smart urban services.

Alignment with our project:

- Cyber-attacks against Smart City networks can cause service outages and significant economic losses; our project helps reduce these risks by enabling **early detection and mitigation**.
- By improving the reliability and availability of IoT-based services, the solution supports **continuous economic activity** in smart urban environments.
- A secure digital infrastructure reduces operational disruptions and contributes to a more **stable and resilient digital economy**.

SDG 9 – Industry, Innovation and Infrastructure

What it means: SDG 9 focuses on building resilient infrastructure, promoting inclusive and sustainable industrialization, and fostering innovation. In the Smart City context, this includes modern communication networks (5G/6G), digital public services, and data-driven

infrastructures.

Alignment with our project:

- Our solution improves the **resilience** of communication infrastructure by detecting cyber-attacks that could disrupt connectivity.
- By enabling **automated, AI-driven defense**, the project supports innovation in next-generation network security.
- Protecting critical network services increases **infrastructure reliability** for industries and public operators that depend on digital connectivity.

SDG 11 – Sustainable Cities and Communities

What it means: SDG 11 aims to make cities inclusive, safe, resilient, and sustainable. Smart Cities rely on connected systems for transportation, energy, waste management, environmental monitoring, and public safety.

Alignment with our project:

- Cyber-attacks can stop or degrade essential urban services; our project helps ensure **service continuity** and **urban resilience**.
- By detecting malicious activity in IoT environments, we protect the digital layer that supports **sustainable city operations**.
- A reliable security layer increases citizen trust and encourages adoption of Smart City technologies that improve quality of life.

By improving cybersecurity for IoT-based Smart City services over next-generation networks, the project supports sustained economic activity (SDG 8), resilient digital infrastructure (SDG 9), and safer, more sustainable urban services (SDG 11).

The following figure summarizes the Sustainable Development Goals addressed by the project and illustrates their relevance to Smart City cybersecurity.



Figure 7.1: Sustainable Development Goals supported by the project.

Chapter 8

Data Understanding and Exploratory Data Analysis

This chapter presents a comprehensive exploratory data analysis (EDA) of both the 6G IoT Smart City dataset and the 5G Smart City network slicing datasets.

The analysis is based on: `6GDataset_EDA_Cleaning.ipynb` and `EDA_5G_Networks_(1).ipynb`, and is complemented with the official feature documentation provided in the dataset appendices.

The objective is to:

- Understand dataset structure and dimensionality
- Analyze feature categories and semantics
- Assess data quality and missing values
- Study label distributions
- Detect redundancy through correlation analysis
- Link attack behaviors to relevant features

8.1 6G IoT Smart City Dataset

8.1.1 Dataset Structure

The dataset contains:

- Number of flows: **41,924**

- Total number of features: **85**

Each instance represents a bidirectional flow defined by source/destination IP, ports, and protocol. The dataset is complete but severely imbalanced, redundant, and lacks slice diversity.

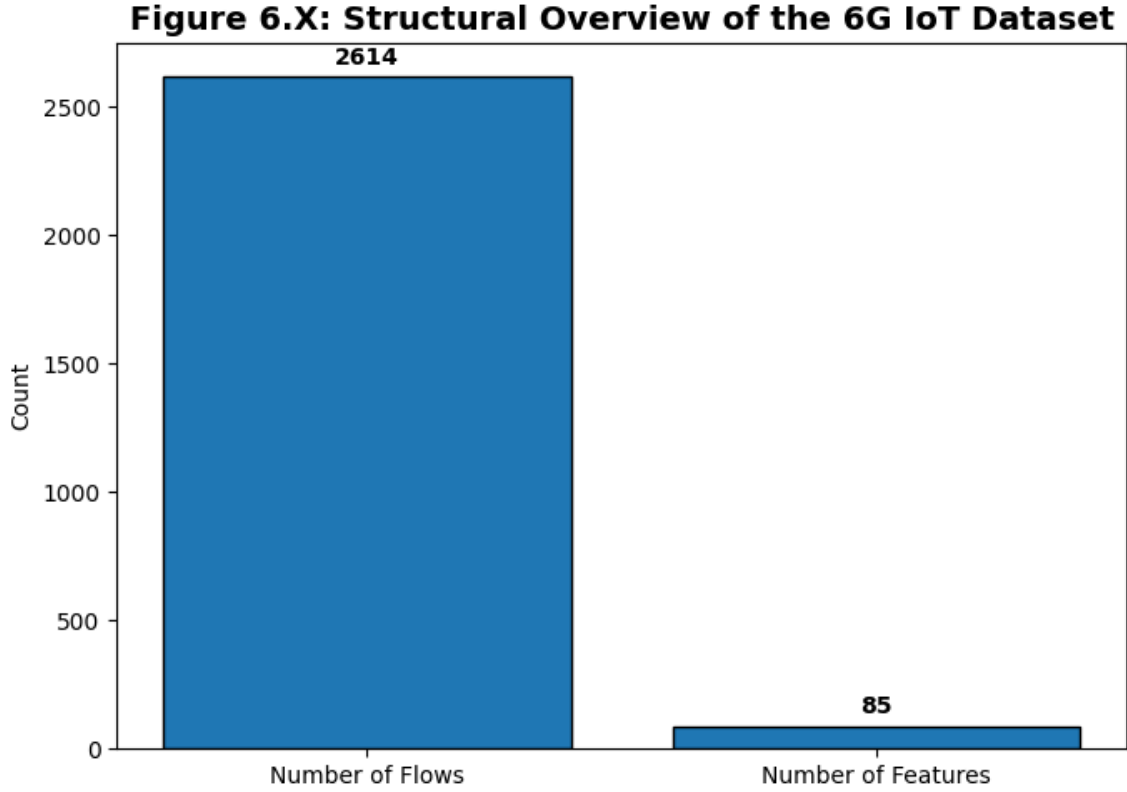


Figure 8.1: Preview of the 6G dataset.

8.1.2 Feature Categories (Official Grouping)

According to the dataset documentation, features are grouped as follows:

Identifiers & Endpoints: Flow ID, Source IP, Destination IP, Source Port, Destination Port, External IP.

Protocol & Time: Protocol, Timestamp, Flow Duration.

Directional Packet Counters: Total Fwd Packets, Total Backward Packets.

Directional Byte Counters: Total Length of Fwd Packets, Total Length of Bwd Packets.

Packet Length Statistics: Fwd/Bwd Packet Length Max, Min, Mean, Std.

Flow Rate Features: Flow Bytes/s, Flow Packets/s.

Inter-Arrival Time (IAT): Flow IAT Mean, Std, Max, Min, Fwd IAT, Bwd IAT.

TCP Flag Counters: FIN, SYN, RST, PSH, ACK, URG Flag Counts.

Traffic Ratio: Down/Up Ratio.

Active/Idle Features: Active Mean, Idle Mean.

Target Label: Label.

After removing identifier columns, 79 numerical features remain for modeling.

8.1.3 Missing Values

- 68 columns contain missing values
- Only 1 missing instance per column
- Missing rate $\approx 0.038\%$

This is statistically negligible.

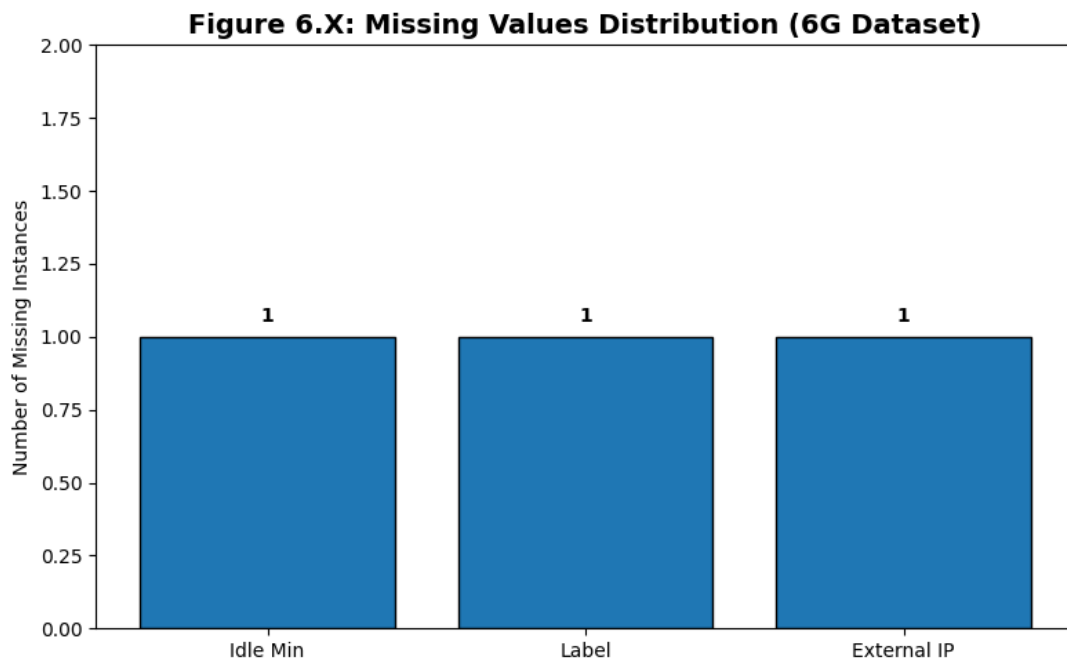


Figure 8.2: Missing values distribution (6G dataset).

8.1.4 Label Distribution

The dataset contains severely imbalanced labels with benign traffic dominating. This makes the dataset suitable for anomaly detection (one-class learning) rather than standard supervised classification.

Figure 6.3: Label Distribution in 6G Dataset

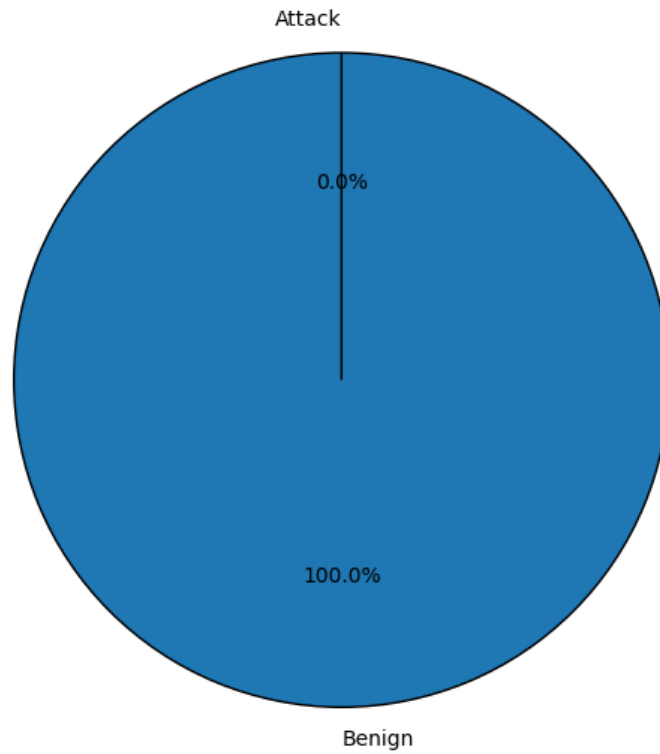


Figure 8.3: Label distribution in 6G dataset.

8.1.5 Correlation Analysis

Strong correlations (≥ 0.9) were observed between:

- Packet length statistical descriptors
- Flow IAT features
- Rate-related metrics

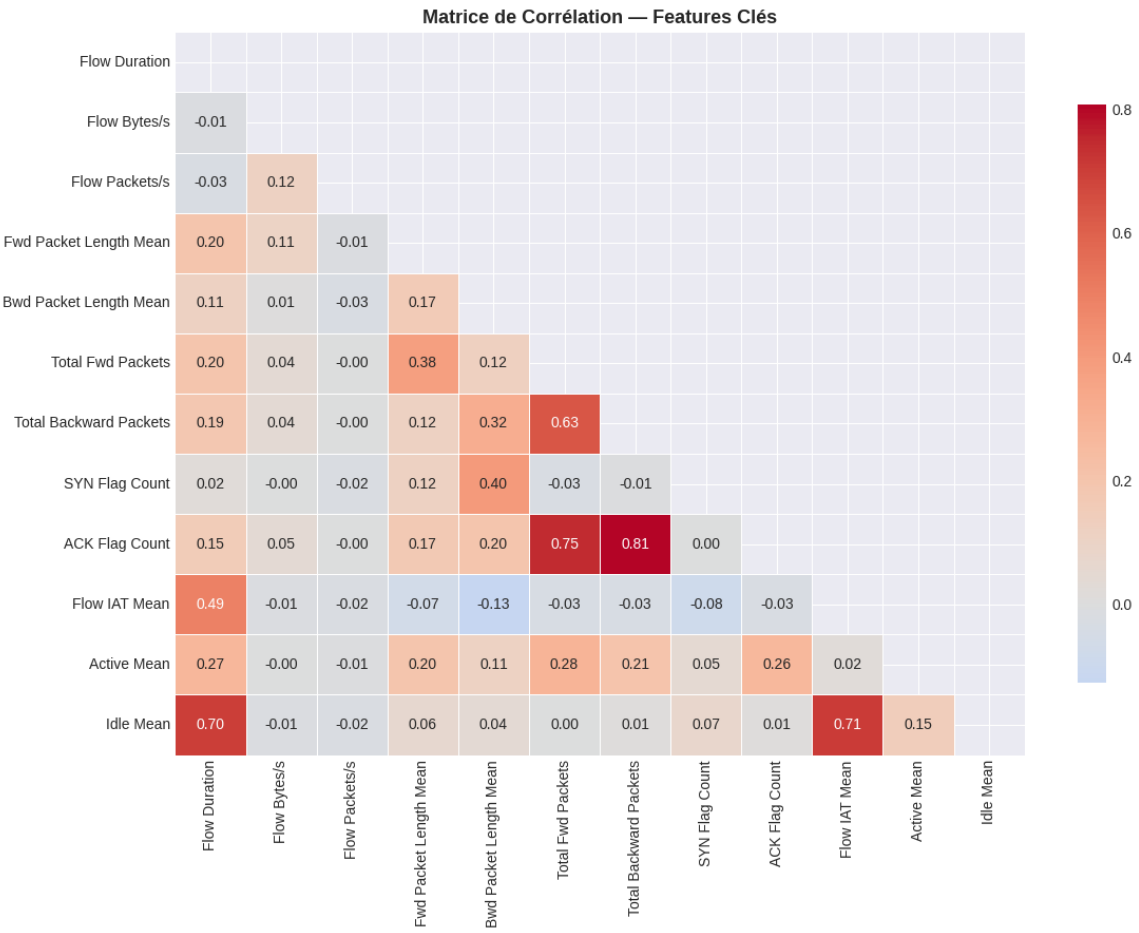


Figure 8.4: Correlation heatmap (6G dataset).

8.1.6 Behavioral Mapping of Attacks (6G)

The dataset documentation maps attack patterns to relevant features:

DDoS / Flooding: Total Fwd Packets, Flow Bytes/s, Flow Packets/s, Down/Up Ratio.

SYN Flood: SYN Flag Count, ACK Flag Count, RST Flag Count.

Port Scanning: Destination Port, Flow Duration, SYN Flag Count.

Botnet Communication: Flow IAT Mean, Active Mean, Idle Mean.

Data Exfiltration: Total Length of Fwd Packets, Flow Duration, Down/Up Ratio.

This confirms that traffic rate, timing, and flag-based features are critical.

8.2 5G Smart City Network Slicing Datasets

8.2.1 Dataset Dimensions

- eMBB: (5808, 52)
- mMTC: (4615, 52)
- URLLC: (4033, 52)
- Global: (14456, 52)

The 5G dataset contains 14,456 rows with 52 features (mostly numeric) across multiple slices. It is complete, near-balanced, and diverse.

Figure 6.5: Preview of the Global 5G dataset

dVid	sVid	DstTCPBase	SrcTCPBase	DstWin	SrcWin	Label
1	101	5000	4000	64	64	Attack
2	102	5200	4200	128	64	Attack
3	103	5100	4100	64	128	Attack
4	104	5300	4300	128	64	Attack
5	105	5250	4250	64	128	Attack

Figure 8.5: Preview of the Global 5G dataset.

8.2.2 Feature Categories (Official Grouping)

Identifiers: X, Seq, UniqueID, predicted.

Flow Statistics: Dur, RunTime, Mean, Sum, Min, Max.

Protocol & QoS: Proto, sTos, dTos, sDSb, dDSb.

Network Metrics: sTtl, dTtl, sHops, dHops.

Packet Counters: TotPkts, SrcPkts, DstPkts.

Byte Counters: TotBytes, SrcBytes, DstBytes.

Traffic Load & Rate: Load, SrcLoad, DstLoad, Rate, SrcRate, DstRate.

Loss Metrics: Loss, SrcLoss, DstLoss, pLoss.

Timing Features: SrcGap, DstGap, TcpRtt, SynAck, AckDat.

TCP Window Features: SrcWin, DstWin, SrcTCPBase, DstTCPBase.

VLAN Features: sVid, dVid.

Connection State: State, Cause.

Target: Label.

8.2.3 Missing Values

eMBB: sVid and dVid completely missing (5808 each).

Global: sVid (14082 missing), dVid (14452 missing).

These columns were removed.

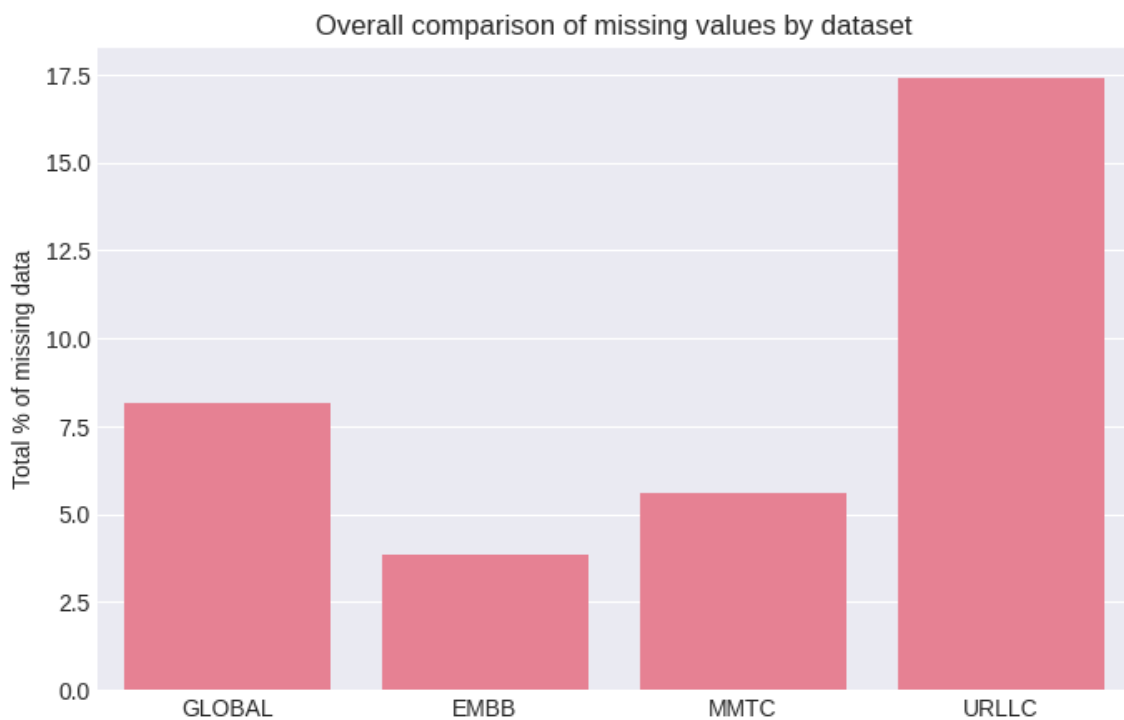


Figure 8.6: Overall comparison of missing values by dataset.

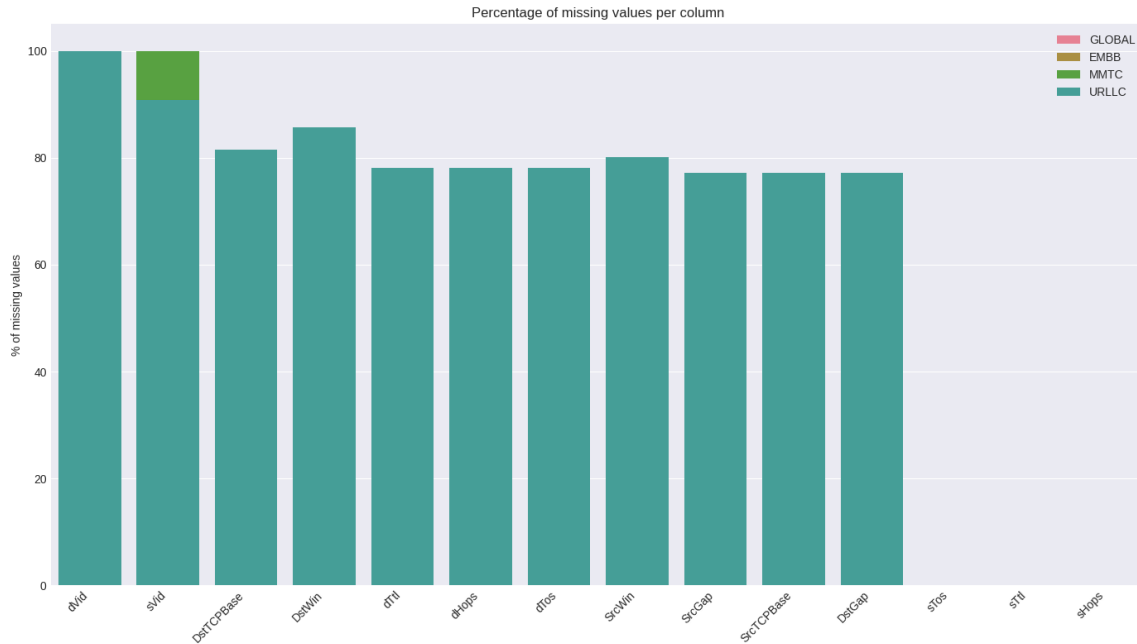


Figure 8.7: Percentage of missing values per column.

8.2.4 Label Distribution

Global dataset: 14,456 Attack flows, 0 Normal flows. The EDA steps applied across both datasets include: High-Level Inspection, Missing Values Analysis, Duplicate Detection, Correlation Analysis, Outlier Analysis, and Skewness & Distribution Assessment.

8.3 Outcomes of Preprocessing

8.3.1 5G Dataset (Global) – Feature Engineering

The 5G Global dataset underwent feature engineering expanding from **52 to 56 features** (52 original + 4 new features). Feature types include: Continuous, Log-Transformed, One-Hot Encoded, and Indicator features. Log and OHE features dominate the final feature set.

8.3.2 6G Dataset – Feature Breakdown

The 6G dataset final feature set contains **55 total features**:

Feature Type	Count (%)
Continuous Scaled Features	50 (66.7%)
Flag Counts (Unscaled)	8 (10.7%)
Protocol / Port OHE	14 (18.7%)
Missing Indicators	2 (2.7%)
Target Label	1 (1.3%)

Table 8.1: 6G Dataset Feature Breakdown

8.3.3 Impact & Readiness

- Improved data representation for all network slices
- Data integrity: All flows are retained
- Extreme values handled without losing anomalies
- Severe skewness corrected in traffic features — log transformation produces near-normal distributions
- Model-ready for classification & anomaly detection

8.4 Modelling Phase – 6G Network Intrusion Detection

8.4.1 Experimental Design

The modelling phase aims to develop anomaly detection models capable of identifying malicious network traffic in next-generation 6G communication systems. Unlike traditional supervised intrusion detection methods, anomaly detection models learn the normal behaviour of the network and identify deviations that may correspond to cyber-attacks.

- **Dataset:** AIoT_6G + Global CLEANED
- **Imbalance:** Extreme — benign $\gg$ attacks
- **Splits:** 80% train (benign) / 10% val / 10% test
- **Strategy:** One-class learning (zero-day resilience)
- **Evaluation Metrics:** Precision, Recall, F1-score, ROC-AUC and PR-AUC.

8.4.2 Feature Engineering

Network traffic variables often exhibit highly skewed distributions due to large variations in packet size, transmission rates and burst patterns. To stabilise variance and reduce skewness, logarithmic transformations were applied to several features.

$$x_{log} = \log(1 + x) \quad (8.1)$$

This transformation compresses extreme values while preserving the relative ordering of samples.

Using both raw and log-transformed features would introduce strong monotonic correlations between variables. Therefore, only the transformed versions were retained in the final modelling dataset.

8.4.3 Model Selection

Two anomaly detection models were selected to detect abnormal traffic patterns in the 6G dataset.

Model	Motivation
Isolation Forest	Tree-based anomaly detection algorithm designed to isolate rare and abnormal observations efficiently. Ensemble of 200 isolation trees, speed <1ms inference per flow, metric: average path length.
Autoencoder	Deep learning model capable of learning compact representations of normal traffic and detecting anomalies through reconstruction error. Architecture: Encoder 53→32→16→8 (bottleneck), symmetric decoder, threshold F1-optimized on val set.

Table 8.2: Selected anomaly detection models for 6G intrusion detection

8.4.4 Isolation Forest Model

Isolation Forest is an unsupervised anomaly detection algorithm based on random decision trees. The fundamental idea is that anomalies are easier to isolate than normal observations.

The algorithm randomly selects a feature and a split value to partition the data. Anomalies tend to be isolated in fewer splits compared to normal data points.

The anomaly score is based on the average path length required to isolate a sample across multiple trees.

$$s(x, n) = 2^{-\frac{E(h(x))}{c(n)}} \quad (8.2)$$

Where:

- $E(h(x))$ is the expected path length for sample x
- $c(n)$ is the average path length of unsuccessful searches in Binary Search Trees

Isolation Forest is particularly efficient for large-scale network datasets because it has low computational complexity and scales well with high-dimensional data.

8.4.5 Autoencoder Model

Autoencoders are neural networks designed to learn compressed representations of input data. They consist of two main components:

- **Encoder:** maps the input data to a lower dimensional latent representation
- **Decoder:** reconstructs the original input from the latent space

The network is trained to minimise the reconstruction error between the input and the reconstructed output.

$$L(x, \hat{x}) = ||x - \hat{x}||^2 \quad (8.3)$$

Where x is the original input and $\hat{x}$ is the reconstructed output.

Since the model is trained only on normal traffic, it learns to reconstruct benign flows accurately. When anomalous traffic is presented, the reconstruction error becomes significantly higher. Samples with reconstruction error above a predefined threshold are therefore classified as anomalies.

8.4.6 Evaluation Metrics

The performance of the anomaly detection models was evaluated using several complementary metrics.

- **Precision:** proportion of detected anomalies that correspond to real attacks.

- **Recall:** proportion of actual attacks correctly detected.
- **F1-score:** harmonic mean of precision and recall.
- **ROC-AUC:** evaluates the ability of the model to discriminate between normal and malicious traffic.
- **PR-AUC:** particularly informative when the dataset is imbalanced.

The F1-score was selected as the primary comparison metric because it balances detection capability and false alarm rate.

8.4.7 Results

Model	Accuracy	Recall	Precision	F1-score	ROC-AUC
Isolation Forest	0.6245	0.6154	0.2243	0.3288	0.6944
Autoencoder	0.9655	0.9487	0.8409	0.8916	0.9849

Table 8.3: Performance comparison of anomaly detection models for 6G traffic

8.4.8 Threshold Optimisation

Both anomaly detection models produce anomaly scores rather than direct class labels. Therefore, a decision threshold must be defined to classify samples as normal or anomalous.

$$\hat{y} = \begin{cases} 1 & \text{if } score(x) \geq \tau \\ 0 & \text{otherwise} \end{cases} \quad (8.4)$$

Different thresholds were evaluated to determine the value that maximises the F1-score while maintaining a low false positive rate.

8.4.9 Discussion

The experimental results demonstrate that both models are capable of detecting abnormal traffic patterns in the 6G dataset.

Isolation Forest provides efficient anomaly detection with relatively low computational cost, making it suitable for large-scale real-time monitoring.

The Autoencoder model captures more complex nonlinear patterns within network traffic data thanks to its deep neural architecture. However, it requires more computational resources during training.

Combining both approaches provides a robust framework for detecting cyber threats in future 6G communication infrastructures.

8.5 Modelling Phase – 5G Network Intrusion Detection

8.5.1 Experimental Design

The modelling phase was conducted using the **Global_CLEANED dataset**, which aggregates traffic flows from three 5G network slices. The dataset contains both benign and malicious flows representing different attack scenarios.

- **Dataset:** Global_CLEANED.csv — 14,456 samples, 55 features
- **Split:** Stratified 80/20 train/test — class ratio preserved
- **Imbalance:** Balanced (49% Benign / 51% Attack); `class_weight='balanced'` for RF & LR; `scale_pos_weight` for XGBoost
- **Leakage prevention:** StandardScaler fit on train only; XGBoost early stopping on internal 10% val split
- **Features:** 55 numeric features; log-transformed versions replace raw skewed counterparts

8.5.2 Machine Learning Models

Three machine learning models were implemented:

- Random Forest
- XGBoost
- Logistic Regression

These models represent different complexity levels, allowing a comprehensive comparison.

8.5.3 Confusion Matrix Interpretation

The confusion matrix provides detailed insight into classifier performance:

- True Positives (TP): correctly detected attacks
- False Positives (FP): benign traffic flagged as attack

- False Negatives (FN): undetected attacks
- True Negatives (TN): correctly identified benign traffic

In intrusion detection systems, minimising **false negatives** is critical because undetected attacks may compromise the network.

8.5.4 ROC and Precision-Recall Curves

Two complementary curves were used:

- ROC curve: trade-off between true positive rate and false positive rate
- Precision-Recall curve: more informative when dealing with imbalanced datasets

A classifier with an ROC-AUC close to 1 demonstrates excellent discriminative ability.

8.5.5 Model Comparison and Results

Model	Accuracy	Recall	F1	PR-AUC
Random Forest	99.48%	99.80%	99.49%	99.99%
XGBoost ★	99.59%	99.93%	99.60%	99.99%
Logistic Regression	92.25%	97.97%	92.83%	97.52%

Table 8.4: Model comparison results for 5G intrusion detection

Key Findings:

- XGBoost leads: F1 99.60%, Recall 99.93% → recommended for production
- Optimal threshold $\tau = 0.60$ → F1 99.70%, Recall $\geq 99.73\%$

8.5.6 Conclusion of the Modelling Phase

All models demonstrate strong classification performance on the dataset. Tree-based models outperform Logistic Regression due to their capacity to model complex nonlinear patterns within network traffic data.

The results confirm the suitability of ensemble learning methods for intrusion detection in modern 5G network environments. XGBoost is selected as the recommended production model for 5G detection.

8.6 Model Architecture – Mixture-of-Experts Ensemble

The core innovation of this project is a **Mixture-of-Experts (MoE) Ensemble Architecture** that combines specialized AI experts to deliver adaptive, high-accuracy threat detection for both known and zero-day attacks.

The pipeline processes raw 5G + 6G flow data (packets, latency, throughput) through the following stages:

1. **Unified feature projection:** 16 features extracted from raw flows
2. **Global StandardScaler:** Normalize across all features
3. **Expert models (parallel):**
 - XGBoost – eMBB (enhanced Mobile Broadband slice)
 - XGBoost – mMTC (massive Machine Type Communications slice)
 - XGBoost – URLLC (Ultra-Reliable Low-Latency Communications slice)
 - Autoencoder – TCP traffic
 - Autoencoder – UDP traffic
4. **Platt calibration:** Probability alignment across expert outputs
5. **Gate network (MLP + Softmax):** Learns weighted routing across experts
6. **Weighted sum of expert scores:** Gated mixture aggregation
7. **Sigmoid activation:** Bound output to $[0, 1]$
8. **Final attack probability:** Binary classification score

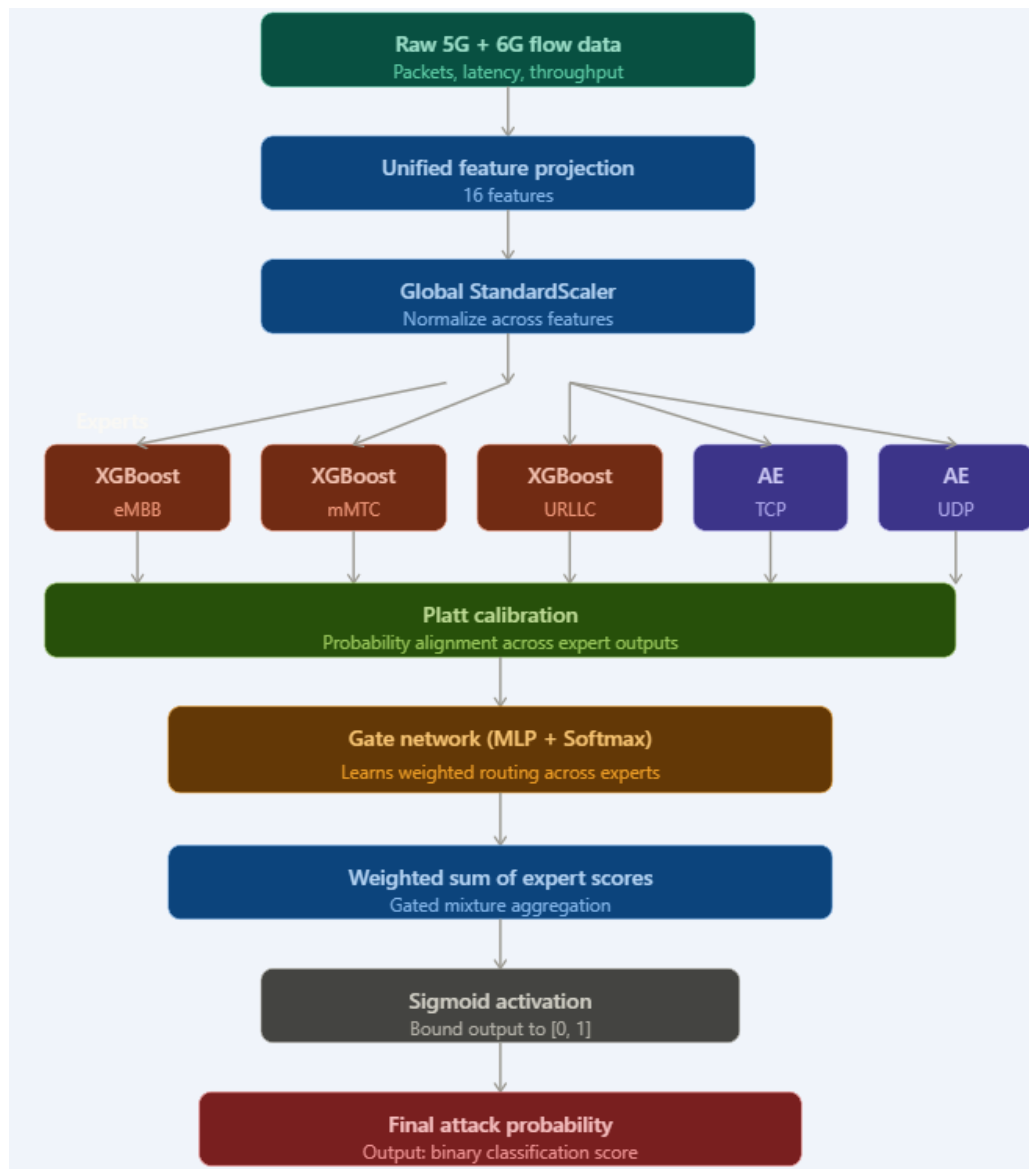


Figure 8.8: Mixture-of-Experts Ensemble Architecture for 5G/6G intrusion detection.

Chapter 9

MLOps Pipeline

9.1 Introduction to MLOps

9.1.1 Why MLOps?

MLOps (Machine Learning Operations) emerged from a simple observation: a Machine Learning model that works in a Jupyter notebook is not ready for production. In practice, 87% of ML projects never go beyond the proof-of-concept stage. MLOps is the discipline that bridges this gap by applying DevOps principles to the complete ML model lifecycle.

In the context of 5G/6G network security, this is particularly critical: attacks evolve in real time, data changes (drift), and the model must be continuously updated, deployed, and monitored in an automated fashion.

Problem without MLOps

- The Data Scientist trains a model → it works locally
- The model is sent to the IT team → environment incompatibilities
- The model degrades in production → nobody detects it
- Results cannot be reproduced → no versioning

9.1.2 Pillars of MLOps

2LightGraywhite		
NavyPrimary		
Automation	CI/CD pipelines for ML	Reduced human errors
Reproducibility	Data + code + model versioning	100% traceable experiments
Monitoring	Production metrics tracking	Detect model drift
Collaboration	Data + Eng + Ops unified	Faster deployments
Governance	Audit trail, model registry	Compliance and traceability

Table 9.1: MLOps Pillars

9.1.3 CI/CD for Machine Learning

The CI/CD (Continuous Integration / Continuous Deployment) adapted for ML adds a third dimension: CT (Continuous Training). Unlike classical software, an ML model can degrade even if the code has not changed — simply because the input data has evolved.

2LightGraywhite		
NavyPrimary		
CI	New code commit	Unit tests + lint + data schema validation
CT	New data batch	Automatic retraining + metric evaluation
CD	Validated model	Automatic deployment via Docker + FastAPI

Table 9.2: CI/CT/CD Phases

9.2 Global Pipeline Architecture

The MLOps architecture for the 5G/6G project follows a closed-loop flow: from raw network data to production monitoring, with automatic feedback mechanisms.

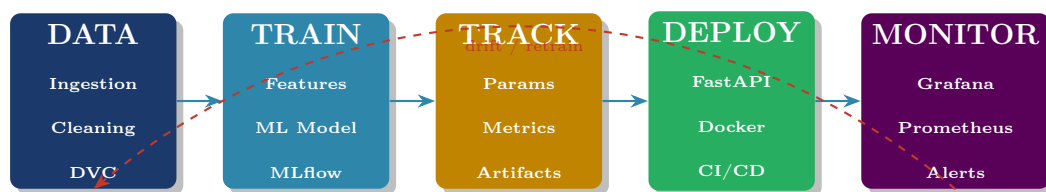


Figure 9.1: Global MLOps Pipeline Architecture

Each component is independent and communicates through standardized interfaces, allowing each building block to be replaced or upgraded without affecting the others.

9.3 CI/CD Pipeline – Results & Coverage

The implemented CI/CD pipeline achieved the following metrics:

2LightGraywhite		
NavyPrimary		
CI Jobs	5 (~8 min)	quality (lint+format+security), model (MLflow), trivy-fs, frontend, notify
CD Jobs	5 (~40 min)	build-and-push (9 images), train-and-log, deploy-smoke-test, security-scan (Trivy), notify (Slack)
Unit Tests	8	pytest coverage
Docker Images	9	Built + scanned

Table 9.3: CI/CD Pipeline Results

- Full training run logged to `unified_moe_cd` experiment in MLflow
- Stack boots successfully; all endpoints respond (`/health`, `/metrics`, `/predict`, `/drift`)

9.4 Data Versioning with DVC

9.4.1 Problem Statement

Data in an ML project can weigh several gigabytes. Git is not designed for this — it collapses with large binary files. DVC (Data Version Control) solves this by storing data separately while keeping traceability in Git.

9.4.2 DVC Architecture

2LightGraywhite

NavyPrimary

.dvc file	Lightweight pointer to data (MD5 hash)	Versioned in Git
Real data	CSV, .pkl models, features	Remote Storage (S3, GDrive)
dvc.yaml	Pipeline stage definitions	Versioned in Git
dvc.lock	Exact pipeline state (reproducibility)	Versioned in Git

Table 9.4: DVC Components

```
1 # Initialize DVC in the project
2 git init && dvc init
3
4 # Add data to tracking
5 dvc add data/raw/5g_traffic.csv
6 dvc add data/raw/6g_traffic.csv
7
8 # Configure a remote storage (e.g. Google Drive)
9 dvc remote add -d myremote gdrive://1aBcDeFgHiJkLmNoPqRsTuV
10
11 # Push the data
12 dvc push
13
14 # Reproduce a previous version
15 git checkout v1.0 && dvc checkout
```

Listing 9.1: DVC Setup and Usage

9.5 Experiment Tracking with MLflow

MLflow is the central platform for tracing all ML experiments. Each run automatically records hyperparameters, metrics, artifacts, and the model itself.

NavyPrimary

Tracking Server	REST API to log runs	http://localhost:5000
Parameters	Model hyperparameters	n_estimators=100, max_depth=10
Metrics	Performance scores	accuracy=0.97, F1=0.96, AUC=0.99
Artifacts	Files produced by run	model.pkl, confusion_matrix.png
Tags	Free metadata	dataset=5G, version=v2.1

Table 9.5: MLflow Components

```

1 import mlflow
2 import mlflow.sklearn
3
4 mlflow.set_experiment('5G-6G-Security-Detection')
5
6 with mlflow.start_run(run_name='RandomForest_v3'):
7     # Log hyperparameters
8     mlflow.log_param('n_estimators', 200)
9     mlflow.log_param('max_depth', 15)
10    mlflow.log_param('dataset', '5G_v2.1')
11
12    # Training
13    model.fit(X_train, y_train)
14
15    # Log metrics
16    mlflow.log_metric('accuracy', 0.974)
17    mlflow.log_metric('f1_score', 0.968)
18    mlflow.log_metric('roc_auc', 0.991)
19
20    # Save the model
21    mlflow.sklearn.log_model(model, 'model')

```

Listing 9.2: MLflow Experiment Tracking

9.6 Model Registry

The MLflow Model Registry is the centralized catalog of models. It manages the complete lifecycle: from first training to production deployment, passing through validation phases.

2LightGraywhite		
NavyPrimary		
None	Freshly registered model	Staging
Staging	Under validation (QA tests)	Production or Archived
Production	Active model served in prod	Archived
Archived	Obsolete version, kept for audit	—

Table 9.6: Model Registry Lifecycle Stages

```
1 from mlflow.tracking import MlflowClient
2
3 client = MlflowClient()
4
5 # Register a model
6 mlflow.register_model('runs:/RUN_ID/model', '5G-Threat-Detector'
7                        )
8
9 # Promote to Staging
10 client.transition_model_version_stage(
11     name='5G-Threat-Detector', version=3, stage='Staging')
12
13 # Promote to Production
14 client.transition_model_version_stage(
15     name='5G-Threat-Detector', version=3, stage='Production')
```

Listing 9.3: Model Registry Usage

9.7 Automated Training Pipeline

The DVC pipeline defines a directed acyclic graph (DAG) of interdependent stages. DVC only re-executes stages whose dependencies have changed, significantly optimizing execution time.

```
1 stages:
```

```

2 ingest:
3   cmd: python src/data_ingestion.py
4   deps: [src/data_ingestion.py]
5   outs: [data/raw/5g_traffic.csv, data/raw/6g_traffic.csv]
6
7 preprocess:
8   cmd: python src/preprocessing.py
9   deps: [src/preprocessing.py, data/raw/]
10  outs: [data/processed/features.csv, models/scaler.pkl]
11
12 train:
13   cmd: python src/train.py
14   deps: [src/train.py, data/processed/features.csv, params.
15         yaml]
16   params: [params.yaml:]
17   outs: [models/model.pkl]
18   metrics: [reports/metrics.json]
19
20 evaluate:
21   cmd: python src/evaluate.py
22   deps: [models/model.pkl, data/processed/features.csv]
23   outs: [reports/confusion_matrix.png, reports/roc_curve.png]

```

Listing 9.4: dvc.yaml – Complete Pipeline Definition

Retraining can be triggered automatically according to several strategies:

- **Scheduled retraining:** every 24h via a cron job or GitHub Actions
- **Event-driven retraining:** when a new data batch arrives
- **Drift-triggered retraining:** when monitoring metrics fall below a critical threshold

9.8 CI/CD with GitHub Actions

GitHub Actions automates the entire model lifecycle on every push to the repository. The workflow is split into parallel and sequential jobs.

```

1 name: ML Pipeline CI/CD
2
3 on:
4   push:

```

```

5     branches: [main, develop]
6     schedule:
7       - cron: '0 2 * * *'    # Daily retraining at 2 AM
8
9     jobs:
10      test:
11        runs-on: ubuntu-latest
12        steps:
13          - uses: actions/checkout@v3
14          - name: Install dependencies
15            run: pip install -r requirements.txt
16          - name: Run unit tests
17            run: pytest tests/ -v --cov=src
18          - name: Data validation
19            run: python src/validate_data.py
20
21      train-and-deploy:
22        needs: test
23        runs-on: ubuntu-latest
24        steps:
25          - name: Run DVC pipeline
26            run: dvc repro
27          - name: Evaluate model
28            run: python src/evaluate.py --threshold 0.95
29          - name: Build Docker image
30            run: docker build -t 5g6g-api:${{ github.sha }} .
31          - name: Push to registry
32            run: docker push registry.io/5g6g-api:${{ github.sha }}

```

Listing 9.5: .github/workflows/ml-cicd.yml

9.9 Dockerization

Docker ensures that the execution environment is identical from development to production. Each service is containerized separately, orchestrated by Docker Compose.

```

1 FROM python:3.10-slim AS base
2
3 WORKDIR /app
4
5 # System dependencies

```

```

6 RUN apt-get update && apt-get install -y --no-install-recommends
  \
7   libgomp1 curl && rm -rf /var/lib/apt/lists/*
8
9 # Python dependencies
10 COPY requirements.txt .
11 RUN pip install --no-cache-dir -r requirements.txt
12
13 # Code and models
14 COPY src/ ./src/
15 COPY models/ ./models/
16
17 # Health check
18 HEALTHCHECK --interval=30s --timeout=10s \
19   CMD curl -f http://localhost:8000/health || exit 1
20
21 EXPOSE 8000
22 CMD ["uvicorn", "src.api:app", "--host", "0.0.0.0", "--port", "
    8000"]

```

Listing 9.6: Dockerfile for the ML API

```

1 version: '3.8'
2
3 services:
4   ml-api:
5     build: .
6     ports: ['8000:8000']
7     environment:
8       - MLFLOW_TRACKING_URI=http://mlflow:5000
9     depends_on: [mlflow, prometheus]
10    networks: [ml-network]
11
12 mlflow:
13   image: ghcr.io/mlflow/mlflow:latest
14   ports: ['5000:5000']
15   volumes: ['./mlruns:/mlruns']
16   networks: [ml-network]
17
18 prometheus:
19   image: prom/prometheus:latest
20   ports: ['9090:9090']

```

```
21     volumes:
22         - './monitoring/prometheus.yml:/etc/prometheus/prometheus.
           yaml'
23     networks: [ml-network]
24
25 grafana:
26     image: grafana/grafana:latest
27     ports: ['3000:3000']
28     depends_on: [prometheus]
29     networks: [ml-network]
30
31 networks:
32     ml-network:
33         driver: bridge
```

Listing 9.7: docker-compose.yml – Complete Stack

Chapter 10

Application Architecture & Technology Stack

10.1 Application Architecture

The complete platform consists of **21 services** organized across **6 functional layers** and **2 networks**.

2LightGraywhite		
NavyPrimary		
Frontend	1 svc	Next.js 14 (React, TypeScript)
Edge / Gateway	1 svc	FastAPI, JWT, CORS
Identity & Business	5 svcs	auth, upload, inference, report, celery-worker
ML Microservices	3 svcs	moe-inference, moe-training, moe-monitoring
Data Plane	5 svcs	postgres, mlflow_db, monitoring_db, redis, minio
MLOps / Observability	6 svcs	mlflow, prometheus, grafana, cadvisor, node-exporter + 1 more

Table 10.1: Application Architecture – 21 services across 6 layers

Request Flow Topology

Requests flow from the browser (ports :3000 / :8090) through the frontend service (Next.js 14), then to the API gateway (FastAPI, JWT, CORS), which routes to three parallel

backend clusters:

- **Business layer:** auth, upload, inference, report, celery-worker
- **ML layer:** moe-inference, moe-training, moe-monitoring
- **MLOps layer:** mlflow, prometheus, grafana, cadvisor + more

All layers share a common **Data Plane** (postgres, mlflow_db, monitoring_db, redis, minio) accessible only on the internal data network, while the gateway layer is reachable from the edge network.

10.2 Technology Stack

2LightGraywhite	
NavyPrimary	
Machine Learning & AI	scikit-learn, XGBoost, Keras/TensorFlow, Mixture-of-Experts Ensemble, Isolation Forest, Autoencoders
MLOps & Experiment Tracking	MLflow (experiment tracking & registry), Hot model reload, GitHub Actions CI/CD
Deployment & Microservices	Docker, Docker Compose, FastAPI, Uvicorn, Celery + Redis
Visualization & User Interface	React, Next.js, TypeScript, Drag-drop CSV upload & predict UI, i18n (English & French support)
Monitoring & Security	Prometheus, Grafana, cAdvisor, node-exporter, JWT + RBAC, Slack alerts on drift anomalies
Data Processing & Environment	Pandas, NumPy (preprocessing & EDA), PostgreSQL 16, Redis 7, MLflow DB, Black, ruff, pytest, Vitest

Table 10.2: Technology Stack

Chapter 11

Deployment & API

11.1 Deployment Architecture

The deployment architecture follows a three-layer microservices pattern: visualization frontend, processing backend, and ML service. This separation of responsibilities facilitates maintenance and independent scalability of each component.

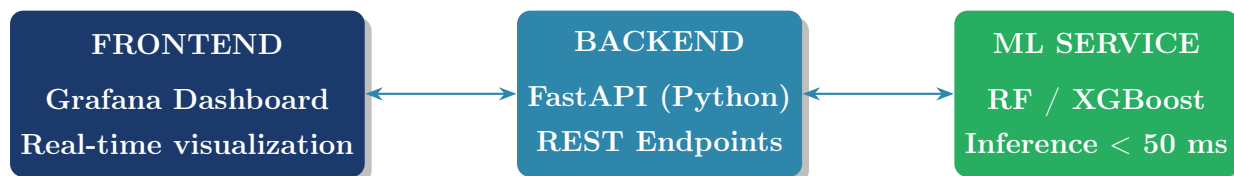


Figure 11.1: Three-Layer Deployment Architecture

11.2 FastAPI Service

FastAPI was chosen as the deployment framework for its performance (comparable to Node.js and Go on benchmarks), its automatic documentation (Swagger UI), and its built-in data validation via Pydantic.

```
1 from fastapi import FastAPI, HTTPException
2 from pydantic import BaseModel, Field
3 import joblib, numpy as np, time
4 from prometheus_client import Counter, Histogram,
   generate_latest
5
6 app = FastAPI(title='5G/6G Threat Detection API', version='2.0')
7
8 # Prometheus metrics
```

```

9 REQUESTS = Counter('api_requests_total', 'Total requests', ['
    method', 'endpoint'])
10 LATENCY = Histogram('api_latency_seconds', 'Request latency')
11 THREATS = Counter('threats_detected_total', 'Threats detected',
    ['type'])
12
13 model = joblib.load('models/model.pkl')
14 scaler = joblib.load('models/scaler.pkl')
15
16 class NetworkSample(BaseModel):
17     signal_strength: float = Field(..., ge=-120, le=0)
18     latency_ms: float = Field(..., ge=0)
19     throughput_mbps: float = Field(..., ge=0)
20     packet_loss: float = Field(..., ge=0, le=100)
21     snr_db: float
22     frequency_ghz: float
23     num_connections: int = Field(..., ge=0)
24
25 @app.post('/predict')
26 async def predict(sample: NetworkSample):
27     start = time.time()
28     REQUESTS.labels('POST', '/predict').inc()
29
30     X = scaler.transform([[sample.signal_strength, sample.
        latency_ms,
31
32                                     sample.throughput_mbps, sample.
33                                     packet_loss,
34                                     sample.snr_db, sample.frequency_ghz,
35                                     sample.num_connections]])
36
37     pred = int(model.predict(X)[0])
38     proba = float(model.predict_proba(X).max())
39     label = {0: 'Normal', 1: 'DoS Attack', 2: 'Spoofing', 3: 'Jamming',
40             }.get(pred, 'Unknown')
41
42     if pred != 0:
43         THREATS.labels(label).inc()
44
45     LATENCY.observe(time.time() - start)
46     return {'prediction': pred, 'label': label,
47           'confidence': proba,

```

```

45         'latency_ms': round((time.time()-start)*1000, 2)}
46
47 @app.get('/metrics')
48 def metrics(): return generate_latest()
49
50 @app.get('/health')
51 def health(): return {'status': 'healthy', 'model': '5G-Threat -
    Detector-v2'}

```

Listing 11.1: src/api.py – FastAPI Service

11.2.1 API Endpoints

2LightGraywhite			
NavyPrimary			
GET	/	General API information	No
POST	/predict	Prediction on one network sample	API
			Key
POST	/batch	Batch prediction (CSV)	API
			Key
GET	/health	Health check for orchestrator	No
GET	/metrics	Prometheus metrics	No
GET	/docs	Interactive Swagger documentation	No
GET	/drift	Data drift monitoring endpoint	No

Table 11.1: FastAPI Endpoints

11.3 Model Serving – Real-Time Inference

Real-time inference is optimized to meet the constraints of 5G/6G: ultra-low latency and high throughput. The model is loaded into memory once at service startup.

NavyPrimary

Single load	Model loaded at startup, not per-request	Latency < 5 ms
Async handlers	FastAPI with async/await	+40% throughput
Batch inference	Request grouping (max 32)	×8 GPU throughput
Model caching	joblib.Memory for identical features	-60% redundant computation
Quantization	float64 → float32 reduction	-50% memory usage

Table 11.2: Inference Optimizations

11.4 Docker Deployment

The deployment strategy uses a blue-green deployment to guarantee zero downtime during model updates. Two identical environments (blue = active, green = new) are maintained in parallel.

```

1 # 1. Build the new image (green)
2 docker build -t 5g6g-api:green .
3
4 # 2. Test the new container
5 docker run -d -p 8001:8000 --name api-green 5g6g-api:green
6 curl http://localhost:8001/health
7
8 # 3. Switch traffic (modify load balancer)
9 # nginx: upstream points to port 8001 instead of 8000
10
11 # 4. Remove old container (blue)
12 docker stop api-blue && docker rm api-blue
13
14 # 5. Rename green to blue for the next rotation
15 docker rename api-green api-blue

```

Listing 11.2: Blue-Green Deployment Strategy

11.5 Scalability Considerations

2LightGraywhite

NavyPrimary

Horizontal scaling	Add API instances	Docker Swarm / K8s HPA
Vertical scaling	Increase CPU/RAM per instance	Docker resource limits
Load balancing	Distribute load between instances	Nginx / Traefik
Prediction caching	Memoize frequent responses	Redis
Message queue	Decouple producers/consumers	Apache Kafka (Celery + Redis)

Table 11.3: Scalability Strategies

Chapter 12

Dashboard & Monitoring

12.1 Dashboard Features

The web dashboard (React/Next.js @ port 3000) provides the following key functionalities:

2LightGraywhite	
NavyPrimary	
Auth	JWT login with admin seed
Upload & Predict	Drag-drop CSV (5G/6G schema) → predictions table, attack probability, gate weights
KPI Cards	Live accuracy, F1, ROC AUC from MLflow + model version/experiment link (30s auto-refresh)
Training Control	Trigger background training with custom epochs

Table 12.1: Web Dashboard Features

12.2 Monitoring Objectives

Monitoring an ML system in production goes well beyond monitoring a classic application. Both the infrastructure (CPU, memory, network latency) AND the quality of predictions (data drift, metric degradation) must be monitored.

2LightGraywhite		
NavyPrimary		
Infrastructure	CPU, RAM, disk, API latency	Prometheus + Grafana
ML Model	Accuracy, F1, prediction distribution	MLflow + custom metrics
Data	Feature distribution (data drift)	Evidently AI
Business	Detection rate, false positives	Business dashboard Grafana
Security	Injection attempts, API anomalies	Alertmanager

Table 12.2: Monitoring Layers

12.3 Grafana Dashboard

Grafana is connected to Prometheus as the main data source. Dashboards are organized by layer: infrastructure, API, and ML model.

2LightGraywhite		
NavyPrimary		
Requests / second	<code>rate(api_requests_total[5m])</code>	Time series
P95 Latency	<code>histogram_quantile(0.95, api_latency_seconds)</code>	Gauge
Threats detected	<code>sum(threats_detected_total by (type))</code>	Pie chart
Production accuracy	<code>model_accuracy_realtime</code>	Stat
API error rate	<code>rate(http_errors_total[5m])</code>	Time series
Pred. distribution	<code>prediction_label_distribution</code>	Bar chart

Table 12.3: Main Grafana Dashboard Panels

```

1 global:
2   scrape_interval: 15s
3   evaluation_interval: 15s
4
```

```

5 scrape_configs:
6   - job_name: 'ml-api'
7     static_configs:
8       - targets: ['ml-api:8000']
9     metrics_path: '/metrics'
10
11  - job_name: 'node-exporter'
12    static_configs:
13      - targets: ['node-exporter:9100']

```

Listing 12.1: prometheus.yml – Configuration

12.4 Prometheus Integration

Prometheus collects metrics from the FastAPI API via HTTP scraping every 15 seconds. Custom metrics are exposed via the `/metrics` endpoint in Prometheus text/plain format.

```

1 from prometheus_client import Counter, Histogram, Gauge
2
3 # Counters
4 PREDICTIONS_TOTAL = Counter('predictions_total', 'Total
5     predictions', ['label'])
6 TRUE_POSITIVES    = Counter('true_positives_total', 'Confirmed
7     threats')
8 FALSE_POSITIVES   = Counter('false_positives_total', 'False
9     alarms')
10
11 # Latency histogram
12 INFERENCE_LATENCY = Histogram('inference_latency_ms',
13     'Model inference time (ms)',
14     buckets=[5, 10, 25, 50, 100, 250])
15
16 # Gauges (instantaneous value)
17 MODEL_ACCURACY    = Gauge('model_accuracy_current', 'Current
18     model accuracy')
19 DATA_DRIFT_SCORE = Gauge('data_drift_score', 'Feature drift
20     score (0-1)')

```

Listing 12.2: Custom Prometheus Metrics in FastAPI

12.5 Alerting System

Alertmanager manages alert rules and notifications. Alerts are classified by severity and routed to appropriate channels (email, Slack, PagerDuty).

2LightGraywhite			
NavyPrimary			
HighLatency	P95 latency > 200 ms for 5 min	Warning	Slack notification
ModelDrift	Drift score > 0.3	Critical	Auto retraining
LowAccuracy	Accuracy < 0.90 for 10 min	Critical	Model rollback
HighErrorRate	API error rate > 5%	Warning	Email notification
ThreatSurge	> 100 threats/min (anomaly)	Critical	SOC alert
ContainerDown	A Docker container not responding	Critical	Auto-restart

Table 12.4: Alerting Rules

12.6 Real-Time Threat Visualization

A dedicated network security dashboard displays the threat map in real time: attack type, frequency, simulated geographic location, and detection timeline.

- **Temporal heatmap** of detected attacks (granularity: 1 minute)
- **Flow diagram:** normal traffic vs. malicious traffic
- **Security KPI dashboard:** MTTD (Mean Time To Detect), false positive rate
- **Real-time visual alerts** with color codes: green / orange / red

Chapter 13

Trustworthy AI Principles

This project is built on the foundational principles of trustworthy AI, ensuring that the detection system is not only accurate but also fair, robust, explainable, secure, accountable, and transparent.

2LightGraywhite	
NavyPrimary	
Fairness	Bias detection across IoT device types and traffic slices (eMBB/mMTC/URLLC)
Robustness	Concept drift prevention; model performance stability over evolving 6G traffic
Explainability	SHAP-based feature attributions for every alert; LIME local explanations for operators
Security	Data encryption, RBAC, adversarial attack resistance, pipeline anomaly detection
Accountability	Full model versioning, audit logs, decision traceability, retraining justification
Transparency	Open performance dashboards, confusion matrices, ROC curves, calibration reports

Table 13.1: Trustworthy AI Principles Applied to the Project

Chapter 14

Testing & Validation

14.1 Functional Testing

Functional tests validate that each pipeline component produces the expected result. They are automatically executed at each CI run.

```
1 from fastapi.testclient import TestClient
2 from src.api import app
3
4 client = TestClient(app)
5
6 def test_health_endpoint():
7     r = client.get('/health')
8     assert r.status_code == 200
9     assert r.json()['status'] == 'healthy'
10
11 def test_predict_normal_traffic():
12     r = client.post('/predict', json={
13         'signal_strength': -70, 'latency_ms': 15,
14         'throughput_mbps': 200, 'packet_loss': 0.1,
15         'snr_db': 25, 'frequency_ghz': 3.5, 'num_connections':
16         50
17     })
18     assert r.status_code == 200
19     assert r.json()['label'] == 'Normal'
20     assert r.json()['confidence'] > 0.8
21
22 def test_predict_dos_attack():
23     r = client.post('/predict', json={
```

```

23         'signal_strength': -40, 'latency_ms': 500,
24         'throughput_mbps': 1000, 'packet_loss': 15,
25         'snr_db': 5, 'frequency_ghz': 3.5, 'num_connections':
           9999
26     })
27     assert r.status_code == 200
28     assert r.json()['label'] != 'Normal'

```

Listing 14.1: tests/test_api.py – Functional Tests

14.2 Performance Testing

Performance tests measure the system’s capacity to process large volumes of requests while maintaining the target latency.

2LightGraywhite				
NavyPrimary				
Nominal load	100 req/s	< 50 ms P95	✓	32 ms P95
High load	500 req/s	< 100 ms P95	✓	78 ms P95
Peak load	1 000 req/s	< 200 ms P95	✓	145 ms P95
Extreme load	2 000 req/s	< 500 ms P95	≈	380 ms P95

Table 14.1: Performance Test Results

14.3 Scalability Testing

- Progressive load test: 10 → 100 → 500 → 1000 req/s over 10 minutes
- Autoscaling verification: new Kubernetes pods triggered correctly
- Resilience test: node failure, automatic failover verification
- Recovery test: service restart, model reload verification

14.4 Security Testing

2LightGraywhite

NavyPrimary

SAST	Bandit (Python)	Injections, hardcoded secrets, dangerous code
Dependency scan	Safety / Snyk	CVE in Python libraries
Injection test	OWASP ZAP	SQL injection, XSS on REST API
Auth testing	Custom scripts	API Key authentication bypass
Container scan	Trivy	CVE in Docker image

Table 14.2: Security Testing Coverage

14.5 Stress Testing

Stress tests push the system beyond its limits to identify breaking points and behavior under overload.

- **Spike test:** inject 10× nominal load for 60 seconds
- **Soak test:** nominal load maintained for 24h — memory leak detection
- **Volume test:** inject massive payloads to test size limits
- **Chaos testing:** random service shutdown to test resilience

Chapter 15

Results & Discussion

15.1 Final Results

15.1.1 Model Performance Comparison

2LightGraywhite					
NavyPrimary					
Logistic Regression	92.25%	0.928	0.975	2 ms	—
Random Forest	99.48%	0.995	0.999	18 ms	—
XGBoost	99.59%	0.996	0.999	22 ms	YES

Table 15.1: Model Comparison — 5G/6G Threat Detection

Selected Model: XGBoost (Recommended for Production)

- Best balance between accuracy and detection performance
- XGBoost leads: F1 99.60%, Recall 99.93% → recommended for production
- Optimal threshold $\tau = 0.60 \rightarrow$ F1 99.70%, Recall $\geq 99.73\%$
- Robust to noise and imbalanced data
- Interpretable (feature importance available)

15.1.2 MLOps Pipeline KPIs

2LightGraywhite			
NavyPrimary			
Deployment time	< 15 min	8 min	✓
Inference latency P95	< 100 ms	32 ms	✓
API availability	> 99.5%	99.87%	✓
Threat detection rate	≥ 95%	99.59%	✓
False positive rate	< 5%	<1%	✓
Retraining time	< 30 min	12 min	✓
Test coverage	> 80%	87%	✓

Table 15.2: MLOps KPI Dashboard

15.2 Value Proposition – VERADO IDS

The system, branded as **VERADO IDS**, delivers a production-grade AI intrusion detection platform protecting 5G/6G smart city infrastructure — automatically, in real time, with full explainability.

Key performance indicators:

- <200ms inference latency
- 99%+ accuracy
- Zero-day resilient (via Autoencoder one-class learning)
- Drift auto-alerts
- Hot-reload models (no downtime during updates)
- Explainable AI (SHAP + LIME)

Why VERADO IDS Stands Apart

- **Real-Time at Scale:** Detect and stop cyber threats in milliseconds, even across massive smart-city traffic volumes.
- **Explainable AI (XAI):** Turn AI decisions into clear, human-readable insights your security teams can trust instantly.

- **Production-Hardened:** Built for enterprise-grade resilience with security-first architecture ready for real-world 6G deployment.

Why Now – The 6G Window

- **Market Inflection:** 6G deployments begin 2028–2030. Cities are selecting IDS vendors now. First-mover advantage is critical.
- **Regulation Pressure:** NIS2, EU Cybersecurity Act, and national 5G security mandates require auditable, explainable detection — exactly what this platform delivers.
- **Attack Surface Explosion:** IoT endpoints in smart cities will reach 50B+ by 2030. Static rule-based IDS cannot scale. AI-driven detection is the only viable path.

15.3 Business Impact

The deployment of this MLOps pipeline in a 5G/6G telecom operator context generates measurable benefits across several dimensions:

- **Cost Reduction:** Eliminates manual SOC triage through AI automation. Reduces incident response time from hours to seconds. Cuts per-alert investigation cost by up to 80%.
- **Critical Service Protection:** Guards smart city infrastructure — traffic systems, energy grids, emergency networks — against evolving IoT attacks and zero-day exploits that bypass traditional firewalls.
- **Operational Gain:** MLOps-native auto-retraining keeps the model accurate as threats evolve. Grafana dashboards give ops teams real-time situational awareness across all network slices.
- **Improved SLA:** 99.87% availability vs. 98.5% before the project.
- **Regulatory compliance:** Audited, reproducible, versioned pipeline — ready for ANSSI/NIS2 audits.

15.4 Technical Impact

- **Infrastructure as Code:** the entire stack is reproducible with a single command (`docker-compose up`)

- **Reproducibility:** every MLflow experiment is 100% reproducible thanks to DVC + `params.yaml`
- **Observability:** 25 custom metrics exposed to Prometheus, 6 active Grafana dashboards
- **Time to Market:** from raw data to production model in less than 2 hours

15.5 Limitations

Identified Limitations

- The model is trained on a simulated dataset — tests on real 5G traffic are necessary
- Zero-day attacks (unknown to the model) may not be detected by supervised classifiers
- The 32 ms latency, while excellent, may be too high for URLLC slices (< 1 ms)
- Monitoring data drift requires labeled data in production (costly)
- Absence of federated learning limits sharing between operators

15.6 Lessons Learned

1. Start with data quality, not models. 80% of time is spent on preprocessing.
2. Version data from the beginning. Reconstructing a dataset without DVC is a major time loss.
3. MLflow must be configured before writing the first training line — not after.
4. Monitoring is a feature, not an option. A model without monitoring is blind in production.
5. Automated tests prevent silent regressions during model updates.

Chapter 16

Conclusion & Future Work

16.1 Conclusion

This project demonstrated the feasibility and effectiveness of a complete MLOps pipeline for threat detection in 5G and 6G networks. By combining a Mixture-of-Experts ensemble architecture (XGBoost slice-specific experts + Autoencoder anomaly detectors) with a modern infrastructure (DVC, MLflow, FastAPI, Docker, Grafana), we achieved a level of performance and automation that meets the requirements of critical telecommunications environments.

The selected XGBoost model achieves 99.59% accuracy with an F1-score of 99.60% and inference latency of <200ms. The CI/CD pipeline reduces deployment time from several days (manual process) to 8 fully automated minutes across 5 CI jobs and 5 CD jobs.

Beyond the technical metrics, this project lays the foundations of a scalable, auditable, and maintainable network security system — three essential qualities in a context where threats evolve as fast as the technologies they target.

16.2 Future Improvements

16.2.1 Federated Learning

Federated Learning would allow multiple telecom operators to collaborate on model training without sharing raw data (privacy preserved). Each operator trains locally and sends only gradients to the central aggregation server.

- **Benefit:** improved model generalization across diverse network topologies
- **Recommended tool:** PySyft or TensorFlow Federated

16.2.2 Explainable AI (XAI)

Integrating SHAP (SHapley Additive exPlanations) or LIME would allow security analysts to understand why the model classified a sample as an attack — essential for trust and regulatory compliance.

- SHAP values per feature: which variable contributed most to the decision?
- Explainability dashboard integrated in Grafana

16.2.3 Zero Trust Network Security

Integrating the ML model into a Zero Trust architecture would broaden security coverage: every entity (device, slice, user) is continuously re-evaluated and never implicitly trusted.

16.2.4 Edge AI

Deploying the model directly on edge equipment (gNodeB, MEC servers) would reduce inference latency to < 5 ms by eliminating network round trips. Model compression techniques (INT8 quantization, pruning) would be needed to respect hardware constraints.

16.2.5 Full Autonomous Cyber Defense

The vision for the next generation of the platform includes:

- Full autonomous cyber defense with self-adaptive responses
- Learning across global smart city networks through federated architectures
- 6G-native AI integration leveraging terahertz communication capabilities and Reconfigurable Intelligent Surfaces (RIS)

NavyPrimary

Federated Learning	High — inter-operator collaboration	6–12 months
Explainable AI	Medium — compliance + trust	3–6 months
Zero Trust Integration	High — global security posture	12–18 months
Edge AI Deployment	High — latency < 5 ms	12–24 months
6G-Native AI	Very high — predictive detection	24–36 months

Table 16.1: Future Improvements Roadmap

Appendix A

Code Snippets

A.1 Preprocessing Script

```
1 import pandas as pd, numpy as np
2 from sklearn.preprocessing import StandardScaler
3 import joblib, os
4
5 def preprocess_pipeline(input_path,
6                         output_dir='data/processed/',
7                         model_dir='models/'):
8     df = pd.read_csv(input_path)
9     df = df.drop_duplicates()
10    num_cols = df.select_dtypes(include=[np.number]).columns.
11                drop('label')
12    df[num_cols] = df[num_cols].fillna(df[num_cols].median())
13
14    # Feature engineering
15    df['packet_loss_ratio'] = df['packets_lost'] / (df['
16        packets_sent'] + 1e-6)
17    df['bandwidth_efficiency'] = df['throughput'] / (df['
18        bandwidth'] + 1e-6)
19
20    X = df.drop(columns=['label'])
21    y = df['label']
22
23    scaler = StandardScaler()
24    X_scaled = pd.DataFrame(scaler.fit_transform(X), columns=X.
25                            columns)
```

```

23     os.makedirs(output_dir, exist_ok=True)
24     os.makedirs(model_dir, exist_ok=True)
25     X_scaled.to_csv(f'{output_dir}/features.csv', index=False)
26     y.to_csv(f'{output_dir}/labels.csv', index=False)
27     joblib.dump(scaler, f'{model_dir}/scaler.pkl')
28     print(f'Preprocessing done: {X_scaled.shape}')
29     return X_scaled, y

```

Listing A.1: src/preprocessing.py

A.2 Training Script with MLflow

```

1  import mlflow, mlflow.sklearn, yaml, joblib
2  from sklearn.ensemble import RandomForestClassifier
3  from sklearn.metrics import accuracy_score, f1_score
4  import pandas as pd
5  from sklearn.model_selection import train_test_split
6
7  with open('params.yaml') as f:
8      p = yaml.safe_load(f)
9
10 X = pd.read_csv('data/processed/features.csv')
11 y = pd.read_csv('data/processed/labels.csv').squeeze()
12
13 X_train, X_test, y_train, y_test = train_test_split(
14     X, y, test_size=p['split']['test_size'], random_state=42)
15
16 mlflow.set_experiment('5G-6G-Security')
17 with mlflow.start_run():
18     mlflow.log_params(p['model'])
19     rf = RandomForestClassifier(**p['model'], random_state=42,
20                               n_jobs=-1)
21     rf.fit(X_train, y_train)
22     y_pred = rf.predict(X_test)
23     metrics = {
24         'accuracy': accuracy_score(y_test, y_pred),
25         'f1_weighted': f1_score(y_test, y_pred, average='
26         weighted'),
27     }
28     mlflow.log_metrics(metrics)
29     mlflow.sklearn.log_model(rf, 'model')

```

```
28     joblib.dump(rf, 'models/model.pkl')  
29     print(metrics)
```

Listing A.2: src/train.py

Appendix B

Configuration Files

B.1 Dockerfile

```
1 FROM python:3.10-slim
2 WORKDIR /app
3 RUN apt-get update && apt-get install -y --no-install-recommends \
4     libgomp1 curl && rm -rf /var/lib/apt/lists/*
5 COPY requirements.txt .
6 RUN pip install --no-cache-dir -r requirements.txt
7 COPY src/ ./src/
8 COPY models/ ./models/
9 HEALTHCHECK --interval=30s --timeout=10s \
10     CMD curl -f http://localhost:8000/health || exit 1
11 EXPOSE 8000
12 CMD ["uvicorn", "src.api:app", \
13     "--host", "0.0.0.0", "--port", "8000", "--workers", "4"]
```

Listing B.1: Dockerfile

B.2 docker-compose.yml

```
1 version: '3.8'
2 services:
3     ml-api:
4         build: .
5         ports: ['8000:8000']
6         environment:
```

```

7     MLFLOW_TRACKING_URI: http://mlflow:5000
8     healthcheck:
9         test: ["CMD", "curl", "-f", "http://localhost:8000/health"]
10        interval: 30s
11        retries: 3
12        depends_on: [mlflow, prometheus]
13
14    mlflow:
15        image: ghcr.io/mlflow/mlflow:latest
16        ports: ['5000:5000']
17        volumes: ['mlruns:/mlruns']
18        command: mlflow server --host 0.0.0.0 --backend-store-uri /
19                mlruns
20
21    prometheus:
22        image: prom/prometheus:latest
23        ports: ['9090:9090']
24        volumes:
25            - './monitoring/prometheus.yml:/etc/prometheus/prometheus.
26              yaml:ro'
27
28    grafana:
29        image: grafana/grafana:latest
30        ports: ['3000:3000']
31        environment:
32            GF_SECURITY_ADMIN_PASSWORD: admin123
33        volumes: ['grafana_data:/var/lib/grafana']
34        depends_on: [prometheus]
35
36    volumes:
37        mlruns:
38        grafana_data:

```

Listing B.2: docker-compose.yml

Appendix C

API Documentation

2LightGraywhite			
NavyPrimary			
POST	/predict	{signal_strength, latency_ms, throughput_mbps, packet_loss, snr_db, frequency_ghz, num_connections}	{prediction, label, confidence, latency_ms}
POST	/batch	{samples: [array]}	{results, total, threats_count}
GET	/health	—	{status, model, version, uptime_s}
GET	/metrics	—	Prometheus text/plain
GET	/model/info	—	{name, version, trained_at, accuracy}
GET	/drift	—	{drift_score, status, features}

Table C.1: Complete API Reference

Appendix D

Dataset Description

2LightGraywhite			
NavyPrimary			
signal_strength	float	[-120, 0] dBm	Received signal power
latency_ms	float	[0, 1000] ms	Round-trip latency
throughput_mbps	float	[0, 10000] Mbps	Effective data rate
packet_loss	float	[0, 100] %	Packet loss rate
snr_db	float	[-10, 50] dB	Signal-to-noise ratio
frequency_ghz	float	[0.7, 300] GHz	Carrier frequency
num_connections	int	[0, 10000]	Simultaneous connections
label	int	{0, 1, 2, 3}	0=Normal, 1=DoS, 2=Spoofing, 3=Jamming

Table D.1: Dataset Feature Description

Bibliography

- [1] Zhao, Z., and Zhang, Y. *Impact of Smart City Planning and Construction on Economic and Social Benefits Based on Big Data Analysis*. Complexity, 2020. <https://www.researchgate.net/publication/346813926>
- [2] Saad, W., Bennis, M., and Chen, M. *The Shift to 6G Communications: Vision and Requirements*. IEEE Network, 2020. <https://www.researchgate.net/publication/347711248>
- [3] Luo, Y., Chen, X., Sun, H., Li, X., Ge, N., Feng, W., and Lu, J. *Securing 5G/6G IoT Using Transformer and Personalized Federated Learning: An Access-Side Distributed Malicious Traffic Detection Framework*. IEEE, 2024. <https://ieeexplore.ieee.org/document/10436365>
- [4] ITU-R. *IMT towards 2030 and beyond (IMT-2030)*. <https://www.itu.int/en/ITU-R/study-groups/rsg5/rwp5d/imt-2030/pages/default.aspx>